

# Architetture Avanzate dei Sistemi di Elaborazione

## **Progetto: Correlation Feature Selection**

## Sommario

|                                                                                                                                |    |
|--------------------------------------------------------------------------------------------------------------------------------|----|
| VERSIONE x86-32 SSE .....                                                                                                      | 4  |
| Primo Approccio in C .....                                                                                                     | 4  |
| Implementazione funzioni in Assembly: CalcoloSommatoriaQuadrato - CalcoloMedia .....                                           | 11 |
| Implementazione funzione in Assembly: calcoloSommatoria e modifica Calcolo Score in C. ....                                    | 17 |
| Ottimizzazioni varie .....                                                                                                     | 23 |
| Implementazione della funzione CalcoloMediaGruppoAssembly in assembly e introduzione loop<br>unrolling.....                    | 25 |
| Considerazioni .....                                                                                                           | 30 |
| Versione x86-64 AVX.....                                                                                                       | 33 |
| Codice in C .....                                                                                                              | 33 |
| Implementazione funzione in Assembly: CalcoloMedia, CalcoloMediaGruppo, CalcoloSommatoria e<br>CalcoloSommatoriaQuadrato ..... | 35 |
| Implementazione funzione in Assembly: sqrtAssembly e calcoloValori_n0n1 .....                                                  | 42 |
| Considerazioni .....                                                                                                           | 45 |
| OpenMp.....                                                                                                                    | 47 |
| Versione a 32 bit e 64 bit.....                                                                                                | 47 |
| Considerazioni .....                                                                                                           | 49 |



## VERSIONE x86-32 SSE

### Primo Approccio in C

L'implementazione dell'algoritmo "Correlation Features Selection" ha avuto inizio implementandolo interamente in linguaggio C. Si è optato per un approccio totalmente modulare; ogni singola funzione, seppur banale, è stata implementata in unità discrete per facilitare l'isolamento degli errori e prepararci per ottimizzazioni future. Inoltre, l'approccio modulare ha fornito un ambiente di sviluppo efficiente all'interno del nostro gruppo. Ciascun membro del gruppo ha potuto così concentrarsi sulla scrittura e sulla ottimizzazione di singole e specifiche funzioni senza dover modificare l'interfaccia, permettendo così ad altri membri di apportare modifiche parallele ad altre parti del codice. Si è favorita, così, una collaborazione parallela e produttiva.

Naturalmente, questa metodologia ha portato alla definizione di "primitive matematiche", cioè funzioni che implementano le operazioni matematiche di base previste nella "Correlation Features Selection", come l'*elevamento a potenza* e la *sommatoria* ed altre, in modo da rendere il codice più chiaro e manutenibile.

GitHub è stato fondamentale per il successo del progetto di gruppo, quindi per la gestione del codice sorgente. Ha agevolato la collaborazione in quanto ha consentito ai membri del gruppo di lavorare su rami separati. Ha permesso di tenere traccia, in maniera semplice e ordinata, delle modifiche apportate al codice ed inoltre della gestione delle diverse versioni del progetto.

Di seguito sono riportate l'elenco e gli screen (del codice C) di queste funzioni primitive, utili alla comprensione e alla definizione delle funzioni più complesse:

- **calcoloSommatoria(...)**, che realizza la sommatoria del prodotto tra: la differenza dell'*i*-esimo valore  $x$  (della feature  $f_x$ ) e  $\mu_x$  (il loro valore medio), e la differenza dell'*i*-esimo valore  $x$  (della feature  $f_{yx}$ ) e  $\mu_y$  (il loro valore medio);
- **calcoloQuadrato(...)**, funzione che calcola il quadrato del valore passato in input;

- **calcoloSommatoriaQuadrato(...)**, che realizza la sommatoria del quadrato della differenza tra l'i-esimo valore  $x$  (della feature  $fx$ ) e la media dei valori assunti dalla feature  $fx$ ;
- **calcoloMedia(...)**, realizza la media dei valori assunti dalla feature  $fx$ ;
- **calcoloMediaGruppo(...)**, realizza la media dei valori assunti dalla feature  $fx$ . I valori considerati devono essere associato al gruppo indicato.

```
type calcoloSommatoria(params *input, type mediaColonna1, type mediaColonna2, int indiceColonna1, int indiceColonna2)
{
    // inizializziamo la somma a 0 perchè è l'identità per l'addizione
    type somma = 0;

    // Iteriamo sui valori delle colonne per calcolare la sommatoria descritta sopra
    for (int i = 0; i < input->N; i++)
        somma += (input->ds[indiceColonna1 + i * input->d] - mediaColonna1) *
                (input->ds[indiceColonna2 + i * input->d] - mediaColonna2);

    //restituzione risultati
    return somma;
}
```

```
Funzioni Esterne usate: -N/A
*/
type calcoloQuadrato(type valore)
{
    return (valore * valore);
}
```

```
type calcoloSommatoriaQuadrato(params *input, type mediaValoriColonna, int indiceColonna)
{
    // inizializziamo la somma a 0 perchè è l'identità per l'addizione
    type somma = 0;

    // iteriamo sui valori della colonna per calcolarne la somma
    for (int i = 0; i < input->N; i++)
        somma += calcoloQuadrato(input->ds[indiceColonna + i * input->d] - mediaValoriColonna); // con k+i*input->d

    return somma;
}
```

```
type calcoloMedia(params *input, int j) // valore medio dei valori assunti della feature e quindi la colonna 'j'-esima della feature
{
    // Inizializzazione dei Dati
    float somma = 0;

    // calcolo numeratore media (valore di mu)
    for (int i = 0; i < input->N; i++)
        somma += input->ds[j + (i * input->d)]; // con j+i*input->d mi sto spostando sui valori della colonna j-esima

    // calcolo Media
    return somma / input->N;
}
```

```

type calcoloMediaGruppo(params *input, int gruppo, int j, int numerosita)
{
    // Inizializzazione dei Dati
    type somma = 0;

    // calcolo numeratore media (valore di mu)
    for (int i = 0; i < input->N; i++)
    {
        if (gruppo == input->labels[i]) {
            somma += input->ds[j + (i * input->d)]; // perché si ha la matrice come vettore riga
        }
    }
    // calcolo Media
    return somma / numerosita;
}

```

Passando ora alla discussione delle funzioni più corpose:

- ***pearsonCorrelationCoefficient(...)***, realizza il calcolo del coefficiente di correlazione di Pearson. In particolare, si è calcolato separatamente il numeratore e denominatore richiamando le opportune primitive matematiche. In fine si è calcolato il rapporto. Il codice è di seguito riportato:

```

type pearsonCorrelationCoefficient(params* input, int indiceColonna1, int indiceColonna2)
{
    // Calcolo media dei valori delle colonne di indice "indiceColonna1" e "indiceColonna2"
    type mediaColonna1 = calcoloMedia(input, indiceColonna1);
    type mediaColonna2 = calcoloMedia(input, indiceColonna2);

    //calcolo numeratore
    type numeratore=calcoloSommatoria(input, mediaColonna1, mediaColonna2, indiceColonna1, indiceColonna2);

    //calcolo denominatore
    type denominatore=sqrt(calcoloSommatoriaQuadrato(input, mediaColonna1, indiceColonna1)) *
    sqrt(calcoloSommatoriaQuadrato(input, mediaColonna2, indiceColonna2));

    //restituzione valori
    return numeratore/denominatore;
}

```

- ***pointBiserialCorrelationCoefficient(...)***, realizza il calcolo del point Biserial Correlation Coefficient. Inizialmente si è calcolati la numerosità dei valori del gruppo 1 e del gruppo 0 e, successivamente, si è proceduto a separatamente i due fattori moltiplicativi. In particolare, il “fattore2” esprima il valore della radice quadrata e il “fattore1” che identifica il rapporto tra la differenza delle medie dei valori assunti dal gruppo 0 e 1, e tra la deviazione standard relativa alla j-esima colonna invocando le rispettive primitive matematiche.

```

type pointBiserialCorrelationCoefficient(params *input, int j)
{
    int n0 = 0;
    int n1 = 0;

    for (int i = 0; i < input->N; i++)
    {
        if (input->labels[i] == 0)
        {
            n0++;
        }
        else
        {
            n1++;
        }
    }
    int n = n0 + n1;

    float num = n0 * n1;
    float den = n * n;

    // secondo fattore
    type fattore2 = sqrt(num / den); // Questo si può calcolare una volta sola e basta.

    // primo fattore
    type fattore1 = (calcoloMediaGruppo(input, 0, j, n0) - calcoloMediaGruppo(input, 1, j, n1)) / calcoloDeviazioneStandard(input, j);

    return fattore1 * fattore2;
}

```

- ***calcoloDeviazioneStandard(...)***, realizza il calcolo della deviazione standard. L'argomento della radice quadrata è stato calcolato separatamente richiamando le opportune funzioni primitive matematiche.

```

type calcoloDeviazioneStandard(params *input, int j)
{
    // Inizializzazione dei dati
    type media = calcoloMedia(input, j);

    type sommatoria = calcoloSommatoriaQuadrato(input, media, j);

    type deviazioneStandard = sqrt((1.0 / (input->N - 1)) * sommatoria); // Valore atteso
    return deviazioneStandard;
}

```

- ***calcoloScore(...)*** e ***calcoloScoreIniziale(...)***, entrambe le funzioni realizzano il calcolo del “merito” dell'insieme di colonne selezionate per la feature i-esima. In particolare, la funzione ***calcoloScoreIniziale()*** è una versione della funzione ***calcoloScore()*** che serve solo quando la cardinalità dell'insieme S è pari a 0, quindi calcola la correlazione della singola colonna rispetto alla variabile dicotomica "c". La funzione ***calcoloScore()***, invece, è la versione che calcola la correlazione della colonna i-esima rispetto, anche, alle altre colonne contenute nell'insieme S (quindi quando la cardinalità di S è maggiore di 0).

```

//funzione merit
type calcoloScore(int indice_feature_corrente, int indice_colonna, params* input)
{
    //aggiorniamo l'insieme di colonne S
    int k=indice_feature_corrente+1; //+1 perch  la cardinalit  dell'insieme S   pari a k, ma dopo che aggiungiamo la
    //colonna corrente per calcolare il nuovo score diventa k+1
    input->out[indice_feature_corrente]=indice_colonna; //S unione f_i (inseriamo la colonna corrente)

    //calcoliamo prima il numeratore
    //calcoliamo il point biserial correlation coefficient
    type rcf_assoluto=0.0;
    for(int i=0;i<k;i++){ //calcoliamo la media dei valori assoluti
        rcf_assoluto+=valoreAssoluto(pointBiserialCorrelationCoefficient(input, input->out[i]));
    }
    type numeratore=rcf_assoluto; //semplifichiamo il fatto che la media sia divisa per k, e si moltiplica per k

    //calcoliamo ora il denominatore
    //calcoliamo il coefficiente di correlazione di pearson tra la colonna corrente e le k-1 colonne gi  presenti
    type rff_assoluto=0.0;
    int z=0;
    for(int i=0;i<k;i++){//calcoliamo la media dei valori assoluti (a sto giro i si ferma prima perch  non vogliamo
    //calcolare il coefficiente di pearson tra la colonna appena inserita e se stessa, ovviamente la colonna appena inserita
    //  in coda)
        for(int j=i+1;j<k;j++){
            z++;
            rff_assoluto+=valoreAssoluto(calcoloPearsonCoefficient(input, input->out[i], input->out[j]));
        }
    }
    type denominatore=(rff_assoluto/z)*(k)*(k-1); //stessa semplificazione di (k-1)/(k-1) di prima

    denominatore+=k;

    //eliminiamo la colonna dall'insieme
    input->out[indice_feature_corrente]=-1;

    //assembliamo
    return numeratore/sqrt(denominatore);
}

```

```

type calcoloScoreIniziale(int indice_feature_corrente, int indice_colonna, params* input)
{
    //aggiorniamo l'insieme di colonne S
    int k=indice_feature_corrente+1; //+1 perch  la cardinalit  dell'insieme S   pari a k, ma dopo che aggiungiamo
    //la colonna corrente per calcolare il nuovo score diventa k+1
    input->out[indice_feature_corrente]=indice_colonna; //S unione f_i (inseriamo la colonna corrente)

    //calcoliamo prima il numeratore
    //calcoliamo il point biserial correlation coefficient
    type rcf_assoluto=0.0;
    rcf_assoluto+=valoreAssoluto(pointBiserialCorrelationCoefficient(input, indice_colonna));

    type numeratore=rcf_assoluto; //semplifichiamo il fatto che la media sia divisa per k, e si moltiplica per k

    //calcoliamo ora il denominatore
    //calcoliamo il coefficiente di correlazione di pearson ad 1, perch  abbiamo una sola colonna nell'insieme S

    //l'unica differenza con l'altra funzione   in queste 2 righe di codice
    type rff_assoluto=1.0;
    type denominatore=rff_assoluto*(k)*(k-1); //non si semplifica il (k-1) perch  non c'  pi  il concetto di media,
    //in quando il coefficiente di pearson   proprio 1

    denominatore+=k;
    //eliminiamo la colonna dall'insieme
    input->out[indice_feature_corrente]=-1;

    //assembliamo
    return numeratore/sqrt(denominatore);
}

```

- **correlationFeatureSelection(...)**,   la funzione principale per la selezione della feature. Per ogni colonna i-esima, calcola il valore di merito dell'insieme S unito l'i-



esima colonna. Alla fine, all'insieme iniziale  $S$ , aggiunge solo la colonna che ha fatto ottenere il valore di merito maggiore. Tale procedura viene ripetuta un numero di volte pari al valore di feature che si vuole estrarre.

```
void CorrelationFeatureSelection(params *input)
{
    //ignorare questa riga, ritornarci dopo aver letto l'intera funzione
    //visto che alla fine sommiamo progressivamente lo score, lo inizializziamo a 0, che è il valore invariante per la somma
    input->sc=0;

    //non ci interessa inizializzare out a tutti "-1" perché di default viene allocato con "k" posizioni disponibili, quindi le sovrascriveremo tutte
    for(int i=0;i<input->k;i++) //Eseguiamo la legge della funzione per "k" volte, quindi vogliamo selezionare "k" colonne
    {
        //l'indice "i" fa riferimento al vettore "out" (vettore degli indici delle colonne selezionate)
        //indice_feature_corrente=i (quindi indica la cardinalità dell'insieme S prima di aggiungere la colonna corrente)
        //l'indice "j" fa riferimento alla colonna su cui calcoliamo lo score
        //indice_colonna=i

        //visto che in pratica stiamo calcolando una funzione di massimo, inizializziamo i valori del confronto per capire qual è il massimo inserendo dei valori
        //fantoccio
        int indice_max=-1;
        type score_max=FLT_MIN;

        //in questo for calcoliamo il massimo merito vero e proprio
        for(int j=0;j<input->d;j++)
        {
            int flag=1; //la flag viene impostata a true di default perché si presuppone che il "j" corrente non sia una colonna già inserita. Se notiamo che gi è
            //stata inserita allora la impostiamo a false (0)
            for(int tmp=0;tmp<i;tmp++)//verifichiamo se la colonna da aggiungere gi è stata aggiunta. Se stiamo aggiungendo la prima colonna, non si entra proprio nel
            //for
            {
                if(input->out[tmp]==j)
                {
                    flag = 0;
                }
            }
            if(i==0) //se l'insieme S è vuoto, non possiamo calcolare la correlazione delle colonne fra di loro, quindi calcoliamo solo la correlazione tra la singola
            //colonna e la variabile dicotomica "c"
            {
                type scoreCalcolato=calcoloScoreIniziale(i,j,input);

                if(score_max<scoreCalcolato)
                {
                    score_max=scoreCalcolato;
                    indice_max=j;
                }
            }
            else //se l'insieme S contiene almeno una colonna, calcoliamo lo score prendendo anche in considerazione il coefficiente di correlazione di pearson
            {
                //if la colonna corrente non appartiene gi all'insieme S
                if(flag) //questo controllo nell'if(i==0) non lo facciamo perché ridondante
                {
                    type scoreCalcolato=calcoloScore(i,j,input);
                    if(score_max<scoreCalcolato)
                    {
                        score_max=scoreCalcolato;
                        indice_max=j;
                    }
                }
            }
            //if else
        }
        //for su j

        //Alla fine del for, aggiungiamo la colonna con lo score massimo all'insieme S; ovvero, aggiungiamo il suo indice al vettore out
        input->out[i]=indice_max;
        //inoltre aggiorniamo lo score, in quanto lo calcoliamo in maniera progressiva
        input->sc=score_max;
    }
    //for su i
}
```

Al fine di quantificare concretamente i benefici derivati dall'implementazione di codice assembly, miglioramenti e ottimizzazioni varie, sono stati condotti test utilizzando questo primo approccio.

Di seguito sono riportati screen rappresentati le specifiche dei test e i risultati ottenuti in termini di tempo e score:

```
Dataset file name: 'test_5000_50_32.ds'  
Labels file name: 'test_5000_50_32.labels'  
Dataset row number: 5000  
Dataset column number: 50  
Number of features to extract: 50  
sc: -1.000000
```

```
CFS time = 0.157 secs  
sc: 0.053388, out: [45,25,7,33,47,]
```

```
Dataset file name: 'test_75000_1350_32.ds'  
Labels file name: 'test_75000_1350_32.labels'  
Dataset row number: 75000  
Dataset column number: 1350  
Number of features to extract: 50  
sc: -1.000000
```

```
CFS time = 88.558 secs  
sc: 0.051457, out: [1265,405,844,199,1337,]
```

```
Dataset file name: 'test_5000_80_32.ds'  
Labels file name: 'test_5000_80_32.labels'  
Dataset row number: 5000  
Dataset column number: 80  
Number of features to extract: 5  
sc: -1.000000
```

```
CFS time = 0.288 secs  
sc: 0.065544, out: [24,41,61,63,53,]
```

```
Dataset file name: 'test_5000_50_32.ds'  
Labels file name: 'test_5000_50_32.labels'  
Dataset row number: 5000  
Dataset column number: 50  
Number of features to extract: 50  
sc: -1.000000
```

```
CFS time = 32.328 secs  
sc: 0.063643, out: [45,25,7,33,47,22,5,31,26,18,6,11,9,27,44,38,32,8,42,14,10,49,29,43,30,48,41,19,0,16,24,21,37,1,13,35,17,15,3,40,39,23,28,46,2,20,12,36,34,4,]
```

```
Dataset file name: 'test_5000_80_32.ds'  
Labels file name: 'test_5000_80_32.labels'  
Dataset row number: 5000  
Dataset column number: 80  
Number of features to extract: 50  
sc: -1.000000
```

```
CFS time = 106.043 secs  
sc: 0.089537, out: [24,41,61,63,53,76,78,26,18,57,62,11,30,34,70,25,16,29,8,37,13,22,49,10,45,9,46,54,27,55,14,19,21,74,42,39,77,23,71,50,68,67,65,40,66,31,32,20,79,12,]
```

Le tempistiche ottenute mettono in evidenza che il codice risulta essere poco prestante.

## Implementazione funzioni in Assembly: CalcoloSommatoriaQuadrato - CalcoloMedia

Durante lo sviluppo delle funzioni in Assembly, abbiamo notato come l'implementazione della matrice memorizzata come un array lineare in row-major order non fosse particolarmente prestante nel caricare il quartetto di dati nelle istruzioni SIMD. In particolare, nell'istruzione SIMD era necessario caricare quattro valori consecutivi della colonna  $i$ -esima. Con un'implementazione della matrice in row-major order, il caricamento di questi quattro valori risulta dispendioso in quanto non occupano quattro celle contigue, bensì quattro celle sparse. Essendo celle sparse, il principio di località spaziale viene meno, non viene sfruttato pienamente, comportando un aumento del numero di cache miss e di conseguenza aumentando il numero di accessi in memoria.

Dunque, abbiamo optato per una trasposizione della matrice in modo tale che la matrice venisse memorizzata come un array lineare in column-major order. Ciò è stato effettuato mediante il codice seguente aggiunto nella funzione *load\_data(...)*:

```
// Siccome load_data viene chiamata sia per caricare .ds che .labels allora questo if mi permette di creare la nuova matrice solo nel caso di .ds
if(cols != 1){
    MATRIX data2 = alloc_matrix(rows, cols);
    for(int j=0; j<cols; j++){
        for(int i=0; i<rows; i++){
            data2[(j * rows) + i] = data[j + (i * cols)];
        }
    }
    return data2;
}
```

Grazie a questa implementazione le funzioni Assembly inserite sono state drasticamente più semplici e più efficienti.

Vediamo adesso le specifiche logiche utilizzate per implementare le funzioni precocemente discusse in assembly.

- ***extern void calcoloSommatoriaQuadrato(...)***, la prima parte della funzione è utilizzata per il settaggio e per la inizializzazione dei registri di lavoro. In questa fase troviamo anche il calcolo dell'indirizzo di memoria di partenza della  $i$ -esima colonna su cui andare a reperire i valori.

```
; mi salvo i valori di utilità
mov EAX, [EBP + input]      ; mi prendo il parametro dell'indirizzo della struct contenente i parametri
mov EBX, [EAX + 20]         ; mi salvo in EBX il valore di 'N'
mov ECX, [EAX]              ; metto in ECX il primo indirizzo del vettore ds

; in ECX va il seguente calcolo = indirizzo di memoria in cui comincia la colonna da valutare
; "indiceColonna" = indirizzoBaseVettoreDS+(indiceColonna*numeroRighe*4)
mov EDX, [EBP + indiceColonna] ; in EDX metto l'indice di colonna da valutare
imul EDX, 4
imul EDX, EBX
add ECX, EDX

; implementazione della sommatoria
; siccome bisogna sommare valori al quadrato allora la strategia è quella di salvare in due registri xmm
; gli stessi valori e poi moltiplicarli
xorps XMM3, XMM3 ; azzeramento registro per somma cumulata
movddup XMM2, [EBP + mediaValoriColonna] ; mi copio quattro volte il valore mediaValoriColonna
shufps XMM2, XMM2, 00000000

; da ora implemento il ciclo
mov EDI, 0

; di seguito si vuole calcolare (numeroValori)/4
mov EDX, 0
mov EDX, EBX ; salvo in EDX il numero di valori contenuti nella colonna
shr EDX, 2 ; shifting sinistro == divisione per 4. Ottengo tutte le quadruple che devo valutare nel ciclo
```

Importante considerare che il numero di iterazioni del ciclo è pari a:  $\text{numero\_di\_righe\_per\_colonna}/4$ . Il valore quattro indica il numero di operandi che un registro SSE può contenere.

Il fulcro del codice è il ciclo. Ad ogni iterazione del ciclo si caricano i valori nel registro XMM0 e siccome si deve calcolare il quadrato se ne fa una copia nel registro XMM1. Come da specifiche della funzione si effettuano le sottrazioni e infine la moltiplicazione valore per valore (dei registri) per ottenere il quadrato. Le somme parziali sono inserite in XMM3 che, successivamente, verranno sommate in via orizzontale tramite l'operazione HADDPS.

```
CSQCiclo:
    cmp EDI, EDX ; ripetiamo il ciclo numeroValori/4
    jge CSQCicloFine
    mov ESI, EDI
    imul ESI, 16
    movaps XMM0, [ECX + ESI] ; prendo i successivi quattro valori
    movaps XMM1, XMM0 ; prendo i successivi quattro valori
    subps XMM0, XMM2 ; input->ds[indiceColonna + (i * input->d)] - mediaValoriColonna)
    subps XMM1, XMM2 ; input->ds[indiceColonna + (i * input->d)] - mediaValoriColonna)
    mulps XMM0, XMM1 ; sto facendo il quadrato. Il risultato viene salvato in XMM0
    addps XMM3, XMM0 ; sto facendo la somma per adesso di 4 a 4 valori, solo alla fine verranno sommatti tutti i 4
    inc EDI
    jmp CSQCiclo

; dobbiamo vedere se devo eseguire il ciclo del residuo
CSQCicloFine:
    imul EDI, 4 ; mi sto già spostando in avanti di EDI*4 valori. Questi valori sono quelli letti dal ciclo

; effettuo le due somme parziali per sommare i 4 valori del registro XMM3
    haddps XMM3, XMM3
    haddps XMM3, XMM3
```

È importante notare la presenza del calcolo degli elementi che dovrà essere gestito dal ciclo dei residui. Caso in cui il numero di righe da considerare non è divisibile per quattro. Se ciò dovesse accadere si effettuerebbero i calcoli singolarmente e non più a gruppi di quattro. Tale calcolo vale per tutte le funzioni. Il calcolo è il seguente: viene moltiplicato il numero di iterazione fatte dal ciclo principale per quattro (poiché i valori vengono considerati a gruppi di quattro), indichiamo questo valore come `numValoriConsiderati`. Successivamente, il ciclo dei residui itera tante volte fin tanto che il `numValoriConsiderati` non è uguale al numero totale di valori da considerare (cioè il numero di righe per la colonna *i*-esima). Attraverso questo processo, in caso di presenza di residui, questi ultimi verranno calcolati singolarmente caricandoli sui registri XMM. (esso è presente in più funzioni e lo screen verrà omissso per leggerezza di lettura).

Infine, il risultato è passato tramite puntatore alla funzione in c.

```

xorps XMM0, XMM0
xorps XMM1, XMM1
xorps XMM4, XMM4

CSQCiclo2:
    cmp EDI, EBX
    jge CSQCicloFine2
    mov ESI, EDI
    imul ESI, 4
    movss XMM0, [ECX+ESI]
    movss XMM1, XMM0
    subps XMM0, XMM2 ; input->ds[indiceColonna + (i * input->d)] - mediaValoriColonna
    subps XMM1, XMM2 ; input->ds[indiceColonna + (i * input->d)] - mediaValoriColonna
    mulps XMM0, XMM1 ; sto facendo il quadrato. Il risultato viene salvato in XMM0
    addps XMM4, XMM0 ; sto facendo la somma per adesso di 4 a 4 valori, solo alla fine verranno sommati tutti i 4
    inc EDI
    jmp CSQCiclo2

CSQCicloFine2:
    addps XMM3, XMM4 ; sommo la somma totale dei valori restituiti con il ciclo a 4 a 4 con la somma dei valori
    ;gestiti dal ciclo di resto

    ; serve per la return della sommatoria
    mov EAX, [EBP + risultato] ; prendo l'indirizzo di memoria su cui salvare il valore della sommatoria
    extractps [EAX], XMM3, 0 ; salvo il valore in posizione 0 di xmm3

; serve per la return della sommatoria
mov EAX, [EBP + risultato] ; prendo l'indirizzo di memoria su cui salvare il valore della sommatoria
extractps [EAX], XMM3, 0 ; salvo il valore in posizione 0 di xmm3

; Sequenza di uscita dalla funzione
pop edi ; ripristina i registri da preservare
pop esi
pop ebx
mov esp, ebp ; ripristina lo Stack Pointer
pop ebp ; ripristina il Base Pointer
ret ; torna alla funzione C chiamante

```

- **extern void calcoloMediaAssembly(...)**, la struttura del codice assembly segue quella della funzione precedente. Ciò che cambia è la logica implementata al suo interno. Ad ogni iterazione del ciclo si sono salvati due quartetti di valori contigui in due registri XMM per poi sommarli. La somma parziale è mantenuta nel registro XMM3. Al

termine del ciclo la somma orizzontale del registro XMM3 si effettuata tramite la funzione HADDPS.

```
cicloMediaAssembly:
; Confronto del contatore
CMP edi, ebx
JGE eventuali_residuiMediaAssembly

; Caricamento dei dati in due quartetti in XMM0 e XMM1
MOV eax, edi
IMUL eax, 16
MOVAPS XMM0, [edx + eax]
INC edi
cmp edi, ebx
JGE eventuali_residuiMediaAssembly

mov eax, edi
IMUL eax, 16
MOVAPS XMM1, [edx + eax]
; Incremento del Contatore
INC edi
ADDPS XMM3, XMM1
ADDPS XMM3, XMM0
JMP cicloMediaAssembly

eventuali_residuiMediaAssembly:
HADDPS XMM3, XMM3
HADDPS XMM3, XMM3

imul edi, 4
mov eax, [ebp+input]
mov ebx, [eax+20]
```

Il dettaglio da analizzare è dovuto al doppio controllo sull'indice (istruzione cmp): ciò avviene nell'eventualità dei residui, in quanto, se fossero presenti, avremmo che senza il controllo duplice dell'indice ad ogni iterazione andrebbe ad incrementarsi di due alla volta. Di conseguenza, senza il secondo controllo, nella seconda fase di caricamento dei dati in XMM1 potrebbe verificarsi uno sfioramento, portando alla considerazione di valori non appartenenti alla colonna corrispondente. Ciò potrebbe addirittura causare una segmentation fault.

Anche in questo caso è stata inserito il calcolo dei residui e il risultato della media è stato inserito nell'apposito puntatore.

Implementate queste due funzioni in assembly si è voluto fare un testing per quantificare il miglioramento ottenuto fino a questo punto.

Di seguito sono riportati screen rappresentati le specifiche dei test e i risultati ottenuti in termini di tempo e score:

|                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>Dataset file name: 'test_5000_50_32.ds' Labels file name: 'test_5000_50_32.labels' Dataset row number: 5000 Dataset column number: 50 Number of features to extract: 5 sc: -1.000000  CFS time = 0.045 secs sc: 0.053386, out: [45,25,7,33,47,]</pre> | <pre>Dataset file name: 'test_75000_1350_32.ds' Labels file name: 'test_75000_1350_32.labels' Dataset row number: 75000 Dataset column number: 1350 Number of features to extract: 5 sc: -1.000000  CFS time = 40.120 secs sc: 0.051457, out: [1265,405,844,199,1337,]</pre> | <pre>Dataset file name: 'test_5000_80_32.ds' Labels file name: 'test_5000_80_32.labels' Dataset row number: 5000 Dataset column number: 80 Number of features to extract: 5 sc: -1.000000  CFS time = 0.075 secs sc: 0.065541, out: [24,41,61,63,53,]</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>Dataset file name: 'test_5000_50_32.ds' Labels file name: 'test_5000_50_32.labels' Dataset row number: 5000 Dataset column number: 50 Number of features to extract: 50 sc: -1.000000  CFS time = 7.655 secs sc: 0.063646, out: [45,25,7,33,47,22,5,31,26,18,6,11,9,27,44,38,32,8,42,14,10,49,29,43,30,48,41,19,0,16,24,21,37,1,13,35,17,15,3,40,39,23,28,46,2,20,12,36,34,4,]</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>Dataset file name: 'test_5000_80_32.ds' Labels file name: 'test_5000_80_32.labels' Dataset row number: 5000 Dataset column number: 80 Number of features to extract: 50 sc: -1.000000  CFS time = 24.982 secs sc: 0.089543, out: [24,41,61,63,53,76,78,26,18,57,62,11,30,34,70,25,16,29,8,37,22,13,49,10,45,9,46,54,27,55,14,19,21,74,42,39,77,23,71,50,68,67,65,40,66,31,32,20,79,12,]</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Notiamo come già solo con queste due sostituzioni di funzioni, da codice C in codice Assembly, i tempi sono notevolmente diminuiti.



## Implementazione funzione in Assembly: calcoloSommatoria e modifica Calcolo Score in C.

- **extern void calcoloSommatoria(...)**, anche qui la struttura del codice assembly segue quella della funzione precedente. La logica della funzione implementa, in linea generale, la logica della funzione calcoloSommatoriaQuadrato().

Ad ogni iterazione del ciclo, vengono caricati all'interno di due diversi registri XMM i quattro valori della colonna x e i quattro valori della colonna j. Ai quattro valori della colonna x viene sottratto il valore medio della colonna x e mentre ai quattro valori della colonna j viene sottratto il valore medio della colonna j (entrambi i valori medi ricevuti come input). Infine, si effettua la moltiplicazione tra i due registri contenente i quattro valori e infine il prodotto viene sommato in un registro intermedio che si occupa di mantenere i risultati parziali.

```
CSCiclo:
    cmp EDI, EDX ; ripetiamo il ciclo numeroValori/4
    jge CSCicloFine
    mov ESI, EDI
    imul ESI, 16
    movaps XMM0, [ECX + ESI] ; prendo i successivi quattro valori della colonna1
    movaps XMM1, [EBX + ESI] ; prendo i successivi quattro valori della colonna2
    subps XMM0, XMM4 ; input->ds[indiceColonna1 + (i * input->d)] - mediaColonna1
    subps XMM1, XMM5 ; input->ds[indiceColonna2 + (i * input->d)] - mediaColonna2
    mulps XMM0, XMM1 ; sto facendo il prodotto dei due fattori. Il risultato viene salvato in XMM0
    addps XMM3, XMM0 ; sto facendo la somma per adesso di 4 a 4 valori, solo alla fine verranno sommati tutti i 4
    inc EDI
    jmp CSCiclo

; dobbiamo vedere se devo eseguire il ciclo del residuo
CSCicloFine:
    imul EDI, 4 ; mi sto già spostando in avanti di EDI*4 valori. Questi valori sono quelle letti dal ciclo

; effettuo le due somme parziali per sommare i 4 valodi del registro XMM3
    haddps XMM3, XMM3
    haddps XMM3, XMM3
```

A termine del ciclo, per calcolare la somma orizzontale del registro di somma si utilizza l'operazione HADDPS. Correttamente viene anche qui scritto il ciclo per la gestione di eventuali residui. Il risultato della sommatoria viene passato tramite puntatore.

- In questa fase del processo di sviluppo dell'applicativo per il calcolo del Correlation Feature Selection, si è deciso di ottimizzare la funzione **calcoloScore(...)**. Questa ottimizzazione, per essere portata a termine, ha comportato una modifica a livello

globale dell'applicativo; infatti, è stato necessario introdurre due nuove variabili globali, "rff\_passato" e "rff\_passato\_momentaneo":

```
251 type rff_passato; //serve per evitare di fare il doppio for per il calcolo dell'rff medio, all'interno della funzione calcolo score.  
252 type rff_passato_momentaneo; //stesso concetto di prima ma questa variabile la usiamo solo per passare il valore del rff appena calcolato
```

noteremo, nel seguito della trattazione, che tali variabili compariranno anche all'interno della funzione ***CorrelationFeatureSelection(...)***, proprio perché la natura di questa ottimizzazione è "globale". L'idea alla base di tale ottimizzazione è quella di eliminare il doppio for, innestato, presente all'interno della funzione del calcolo del merito, che ha lo scopo di calcolare la media dei vari coefficienti di Pearson (in valore assoluto). Nel momento in cui viene inserita una nuova colonna nell'insieme delle feature, si calcola la media dei coefficienti di Pearson, ma questi ultimi si calcolano tramite tutte le possibili coppie (senza ripetizione e non ordinate) di colonne presenti nell'insieme di feature. È ovvio che tale calcolo richiede un doppio for innestato. Allora, come è possibile eliminare tale struttura? Sfruttando la natura della funzione ***CorrelationFeatureSelection(...)***; ovvero, visto che inizialmente l'insieme delle feature è vuoto, e visto che è la funzione stessa a popolarlo inserendo una colonna per volta, possiamo memorizzarci ad ogni nuova espansione dell'insieme delle feature (ad ogni nuovo inserimento di una colonna) la somma dei coefficienti di Pearson calcolati tra le colonne presenti nell'insieme, così che quando ne inseriremmo un'altra andremo a calcolare il coefficiente di Pearson soltanto tra la nuova colonna inserita e le colonne già presenti nell'insieme di feature. Così facendo si elimina la struttura del doppio for innestato. Riportiamo di seguito una serie di descrizioni istantanee del calcolo appena descritto, indicando con  $rf_{ij}$  il coefficiente di Pearson calcolato tra la colonna di indice  $i$  e la colonna di indice  $j$ :

**iterazione 1)** A partire dall'insieme di feature vuoto inseriamo una colonna, di cui non dobbiamo calcolare l'rff, in quanto esso è una misura della ridondanza; dunque, non avrebbe senso calcolare la ridondanza di un elemento rispetto a sé stesso. Ipotizziamo che alla fine di tale fase l'insieme di feature **S** conterrà la colonna di indice **45**;

**iterazione 2)** Ipotizziamo che in tale fase venga inserita la colonna di indice **25**, allora verrà calcolato  $rf_{45f_{25}}$ . Possiamo memorizzare tale valore.

**Iterazione 3)** Ipotizziamo che in tale fase venga inserita la colonna di indice **7**, allora basterà calcolare  $rf_{45f_7}$  e  $rf_{25f_7}$ , poi sommarvi  $rf_{45f_{25}}$  e dividere per tre. Memorizziamo la somma appena descritta.

**Iterazione 4)** Ipotizziamo che in tale fase venga inserita la colonna di indice **33**, dunque basterà calcolare  $rf_{45}f_{33}$ ,  $rf_{25}f_{33}$  e  $rf_7f_{33}$ , sommarvi  $rf_{45}f_{25}$ ,  $rf_{45}f_7$  e  $rf_{25}f_7$  e dividere per sei. Possiamo memorizzare tale somma.

**Si itera tale schema per ogni iterazione.**

Per quanto questo schema possa risultare intuitivo e “facile” da applicare, in realtà non è così, per due motivi principali:

1. Alla fine di ogni iterazione ciò che calcoliamo è l'intera media e non la singola somma (dunque non solo il numeratore);
2. Alla fine di ogni iterazione il numero per cui dividere non si conosce.

Per ovviare al primo problema non possiamo far altro che memorizzare la somma di interesse in una variabile globale (`rff_passato_momentaneo`) prima di calcolare la media.

Per ovviare al secondo problema si analizza, alla *i*-esima iterazione, il numero per cui dividere:

**iterazione I)** non si divide, perché vi è solo una colonna;

**iterazione II)** si divide per **1**;

**iterazione III)** si divide per **3**;

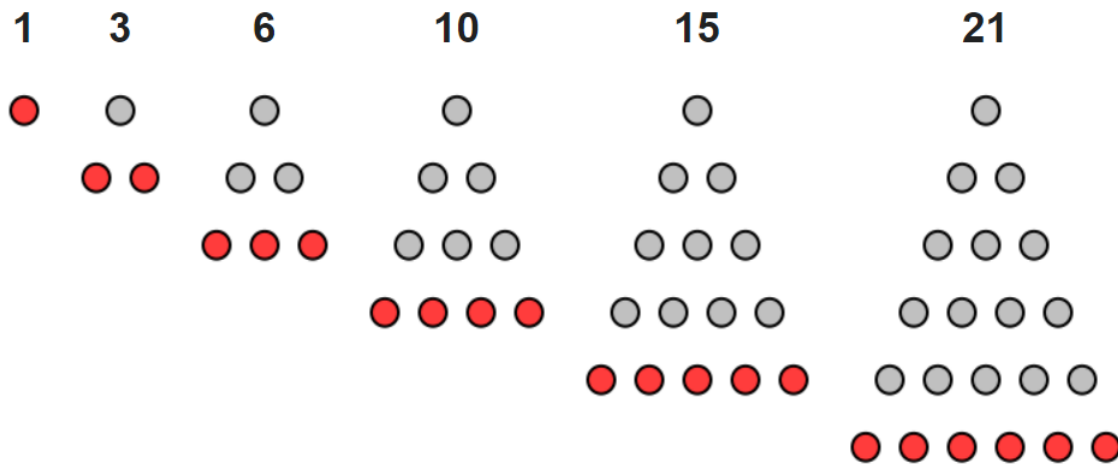
**iterazione IV)** si divide per **6**;

**iterazione V)** si divide per **10**;

notiamo che questa successione di numeri è la più nota **serie triangolare**. Quest'ultima è caratterizzata dalla formula  $T_n = \frac{n \cdot (n+1)}{2}$ . Visto che la nostra serie inizia dalla seconda iterazione, e non dalla prima, poniamo  $n = (k - 1)$ , dove *k* è la *k*-esima colonna che inseriamo, dunque la *i*-esima iterazione. Dunque la nostra serie triangolare sarà esplicitata nella seguente forma:

$$T_n = Denominatore = \frac{(k - 1) \cdot k}{2}$$

Ricordiamo, per pura nozione culturale, che la serie triangolare è così chiamata perché usata per calcolare il numero di elementi che possiamo inserire all'interno di un triangolo equilatero la cui area è stata opportunamente discretizzata, come rappresentato nella seguente immagine:



Ritornando, ora, alla funzione ***calcoloScore(...)***, possiamo analizzarne il codice:

```
type calcoloScore(int indice_feature_corrente, int indice_colonna, params* input)
{
    //aggiorniamo l'insieme di colonne S
    int k=indice_feature_corrente+1; //+1 perch  la cardinalit  dell'insieme S   pari a k, ma dopo che aggiungiamo la colonna corrente
    input->out[indice_feature_corrente]=indice_colonna; //S unione f_i (inseriamo la colonna corrente)

    //calcoliamo prima il numeratore
    //calcoliamo il point biserial correlation coefficient
    type rcf_assoluto=0.0;
    for(int i=0;i<k;i++) { //calcoliamo la media dei valori assoluti
        rcf_assoluto+=valoreAssoluto(pointBiserialCorrelationCoefficient(input, input->out[i]));
    }
    type numeratore=rcf_assoluto; //semplifichiamo il fatto che la media sia divisa per k, e si moltiplica per k

    //calcoliamo ora il denominatore
    //calcoliamo il coefficiente di correlazione di pearson tra la colonna corrente e le k-1 colonne gi  presenti
    type rff_assoluto=0.0;
    for(int i=0;i<k-1;i++) { //calcoliamo la media dei valori assoluti (a sto giro i si ferma prima perch  non vogliamo calcolare il (
        rff_assoluto+=valoreAssoluto(calcoloPearsonCoefficient(input, input->out[i], input->out[indice_feature_corrente]));
    }
    //invece di calcolare l'rff tra tutte le coppie di colonne selezionate, lo calcoliamo tra le k-1 gi  presenti e la k-esima appena
    rff_assoluto+=rff_passato;

    //aggiorniamo l'rff passato per la prossima volta che questa funzione verr  richiamata (in realt  poi decider  la funzione chiam
    rff_passato_momentaneo=rff_assoluto;

    //ora serve calcolare la media, dunque ci serve sapere il numero per cui dividere, tale numero lo calcoliamo tramite la serie tr
    type den=((k-1)*k)/2;
    type denominatore=(rff_assoluto/den)*(k)*(k-1); //stessa semplificazione di (k-1)/(k-1) di prima

    denominatore+=k;

    //eliminiamo la colonna dall'insieme
    input->out[indice_feature_corrente]=-1;

    //assembliamo
    return numeratore/sqrt(denominatore);
}
```

È immediato capire che la variabile globale ***rff\_passato*** rappresenta la somma progressiva, degli rff della iterazione precedente, da aggiungere. Analizziamo, in fine, il punto del codice in cui le due variabili globali vengono modificate. In particolare, osserviamo la funzione ***CorrelationFeatureSelection(...)***:

```

int indice_max=-1;
type score_max=-1.0;
type rff_max=0.0; //va inizializzato a 0.0 cos da sovrascrivere l'rff_passato ed iniziarlo alla prima iterazione

//in questo for calcoliamo il massimo merito vero e proprio
for(int j=0;j<input->d;j++) {
    int flag=1; //la flag viene impostata a true di default perch si presuppone che il "j" corrente non sia una colonna gi
    if(vettore[j] == 1)
        flag = 0;
    if(i==0) { //se l'insieme S vuoto, non possiamo calcolare la correlazione delle colonne fra di loro, quindi calcoliamo

        type scoreCalcolato=calcoloScoreIniziale(i,j,input);

        if(score_max<scoreCalcolato) {
            score_max=scoreCalcolato;
            indice_max=j;
        }
    } else { //se l'insieme S contiene almeno una colonna, calcoliamo lo score prendendo anche in considerazione il coeffi
        //if la colonna corrente non appartiene gi all'insieme S
        if(flag) { //questo controllo nell'if(i==0) non lo facciamo perch ridondante
            type scoreCalcolato=calcoloScore(i,j,input); //passiamo per riferimento questa variabile perch ci serve solo pe
            if(score_max<scoreCalcolato) {
                score_max=scoreCalcolato;
                indice_max=j;
            }
        }
    }
}

//Alla fine del for, aggiungiamo la colonna con lo score massimo all'insieme S; ovvero, aggiungiamo il suo indice al vettore
input->out[i]=indice_max;
vettore[indice_max] = 1;
//inoltre aggiorniamo lo score, in quanto lo calcoliamo in maniera progressiva
input->sc=score_max;
//aggiorniamo l'rff passato con quello calcolato in corrispondenza dello score massimo
rff_passato=rff_max;
}
}

```

Quindi, nella funzione **calcoloScore(...)** modifichiamo ad ogni iterazione la variabile globale **rff\_passato\_momentaneo**, esso, a sua volta, viene memorizzato nella variabile locale alla funzione **CorrelationFeatureSelection(...)** in corrispondenza della colonna che ha generato lo score migliore (così da non perderne il valore quando la prossima iterazione di **calcoloScore(...)** la sovrascriverà). In fine, nella funzione **CorrelationFeatureSelection(...)**, modifichiamo la variabile globale **rff\_passato** con l'**rff\_passato\_momentaneo** calcolato in corrispondenza della colonna che ha generato il massimo score. Seppure la nomenclatura di queste due variabili globali risulti infelice, non bisogna confondersi e ricordare che esse memorizzano la somma dei coefficienti di Pearson in valore assoluto, e non la media degli stessi.

I miglioramenti ottenuti sono di seguito mostrati dai seguenti test:

```
Dataset file name: 'test_5000_50_32.ds'  
Labels file name: 'test_5000_50_32.labels'  
Dataset row number: 5000  
Dataset column number: 50  
Number of features to extract: 5  
sc: -1.000000  
  
CFS time = 0.025 secs  
sc: 0.053388, out: [45,25,7,33,47,]
```

```
Dataset file name: 'test_75000_1350_32.ds'  
Labels file name: 'test_75000_1350_32.labels'  
Dataset row number: 75000  
Dataset column number: 1350  
Number of features to extract: 5  
sc: -1.000000  
  
CFS time = 21.195 secs  
sc: 0.051456, out: [1265,405,844,199,1337,]
```

```
Dataset file name: 'test_5000_80_32.ds'  
Labels file name: 'test_5000_80_32.labels'  
Dataset row number: 5000  
Dataset column number: 80  
Number of features to extract: 5  
sc: -1.000000  
  
CFS time = 0.032 secs  
sc: 0.065544, out: [24,41,61,63,53,]
```

```
Dataset file name: 'test_5000_50_32.ds'  
Labels file name: 'test_5000_50_32.labels'  
Dataset row number: 5000  
Dataset column number: 50  
Number of features to extract: 50  
sc: -1.000000  
  
CFS time = 0.650 secs  
sc: 0.063643, out: [45,25,7,33,47,22,5,31,26,18,6,11,9,27,44,38,32,8,42,14,10,49,29,43,30,48,41,19,0,16,24,21,37,1,13,35,17,15,3,40,39,23,28,46,2,20,12,36,34,4,]
```

```
Dataset file name: 'test_5000_80_32.ds'  
Labels file name: 'test_5000_80_32.labels'  
Dataset row number: 5000  
Dataset column number: 80  
Number of features to extract: 50  
sc: -1.000000  
  
CFS time = 1.737 secs  
sc: 0.089537, out: [24,41,61,63,53,76,78,26,18,57,62,11,30,34,70,25,16,29,8,37,13,22,49,10,45,9,46,54,27,55,14,19,21,74,42,39,77,23,71,50,68,67,65,40,66,31,32,20,79,12,]
```

## Ottimizzazioni varie

In una seconda fase abbiamo notato la presenza di un errore di ottimizzazione. In particolare nella funzione *pointBiserialCorrelationCoefficient()* vi era un'istruzione for che effettuava il calcolo di n0 e n1 ogni volta che la funzione veniva invocata.

Tale calcolo non deve essere fatto ad ogni chiamata in quanto questi due fattori dipendono esclusivamente dal vettore Label e non dalla i-esima colonna presa in considerazione.

Quindi, risulta ovvio che tale calcolo può essere implementato solo una volta, in particolare prima di cominciare a trovare le k-esima feature, in particolare all'inizio della funzione CorrelationFeature. Tale approccio ha richiesto l'inserimento di due variabili globali n0 e n1.

I miglioramenti ottenuti in termini di tempo sono di seguito riportati:

|                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>Dataset file name: 'test_5000_80_32.ds' Labels file name: 'test_5000_80_32.labels' Dataset row number: 5000 Dataset column number: 80 Number of features to extract: 5 sc: -1.000000  CFS time = 0.040 secs sc: 0.065544, out: [24,41,61,63,53,]</pre>                                                                                                                                       | <pre>Dataset file name: 'test_75000_1350_32.ds' Labels file name: 'test_75000_1350_32.labels' Dataset row number: 75000 Dataset column number: 1350 Number of features to extract: 5 sc: -1.000000  CFS time = 21.163 secs sc: 0.051456, out: [1265,405,844,199,1337,]</pre> | <pre>Dataset file name: 'test_5000_80_32.ds' Labels file name: 'test_5000_80_32.labels' Dataset row number: 5000 Dataset column number: 80 Number of features to extract: 5 sc: -1.000000  CFS time = 0.032 secs sc: 0.065544, out: [24,41,61,63,53,]</pre> |
| <pre>Dataset file name: 'test_5000_50_32.ds' Labels file name: 'test_5000_50_32.labels' Dataset row number: 5000 Dataset column number: 50 Number of features to extract: 50 sc: -1.000000  CFS time = 0.643 secs sc: 0.063643, out: [45,25,7,33,47,22,5,31,26,18,6,11,9,27,44,38,32,8,42,14,10,49,29,43,30,48,41,19,0,16,24,21,37,1,13,35,17,15,3,40,39,23,28,46,2,20,12,36,3]</pre>             |                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                             |
| <pre>Dataset file name: 'test_5000_80_32.ds' Labels file name: 'test_5000_80_32.labels' Dataset row number: 5000 Dataset column number: 80 Number of features to extract: 50 sc: -1.000000  CFS time = 1.735 secs sc: 0.089537, out: [24,41,61,63,53,76,78,26,18,57,62,11,30,34,70,25,16,29,8,37,13,22,49,10,45,9,46,54,27,55,14,19,21,74,42,39,77,23,71,50,68,67,65,40,66,31,32,20,79,12,]</pre> |                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                             |

Inoltre, si è pensato di implementare il calcolo di n0 e n1 in Assembly (***extern void calcoloValori\_n0n1(...)***), ma non ha prodotto notevoli miglioramenti.

In Assembly la funzione ***extern void calcoloValori\_n0n1(...)*** è stata implementata secondo questo ragionamento: dato che il vettore Labels è costituito interamente da uni e da zero si è optato per sommare unicamente i valori pari ad uno. Quindi nel corpo del ciclo si sono

caricati quattro valori consecutivi in un registro XMM effettuando dopo la somma parziale in un secondo registro.

Al termine del ciclo nel secondo registro utilizzato, erano presenti le somme parziali del numero di valori pari ad uno. Con l'istruzione HADDPS si è ottenuto l'intero quantitativo di valori pari ad uno. Per calcolare il numero di valori pari a zero si è effettuata una differenza tra il totale dei valori di Labels denotato dal valore di 'N' e il quantitativo di uni.

```
ciclo_calcoloValorin0n1:
    cmp edi,ebx
    JGE residui_calcoloValorin0n1

    ;caricamento dei valori in XMM0
    MOV edx, edi
    IMUL edx, 16
    MOVAPS XMM0, [ecx + edx]
    ADDPS XMM1,XMM0
    inc edi

    JMP ciclo_calcoloValorin0n1

residui_calcoloValorin0n1:
    HADDPS XMM1,XMM1
    HADDPS XMM1,XMM1
```

```
fine_calcoloValorin0n1:
    mov esi, [ebp + 12] ; Carica il puntatore al risultato di n0
    mov ebx, [ebp + 16] ; Carica il puntatore al risultato di n1
    CVTSS2SI ecx, XMM1 ;Converte il float in intero
    mov [ebx],ecx ; Salva il risultato di n1 sul puntatore associato

    ;calcoliamo il valore n1
    mov edx, [eax+20] ;valore di N
    sub edx,ecx ; Totale valori-Totale valori di n1
    mov [esi],edx
```

Ulteriore modifica apportata è stata quella di implementare la funzione ***sqrAssembly()*** (nativamente offerta da C) in assembly. Non ci sono stati degli evidenti miglioramenti riguardo l'ottimizzazione.

```
movss xmm0, [ebp+8] ;prendiamo il parametro su cui calcolare la radice
mov ebx, [ebp+12] ;prendiamo il riferimento in cui salvare il risultato
;
; corpo della funzione
;

sqrtps xmm0, xmm0 ;calcoliamo la radice quadrata
movss dword [ebx], xmm0 ;restituisco il valore calcolato
```



## Implementazione della funzione `CalcoloMediaGruppoAssembly` in assembly e introduzione loop unrolling.

- **`extern void calcoloMediaGruppoAssembly(...)`**, anche qui la struttura del codice assembly segue quella delle funzioni precedenti. La logica della funzione implementa, segue questo ragionamento: sapendo che il vettore di Labels è costituito da valori posti a uno e a zero, che rappresentano rispettivamente il gruppo uno ed il gruppo zero, si è sfruttato ciò come maschera.

In particolare, ad ogni iterazione del for si caricava un quartetto di valori del Labels e si effettuava una moltiplicazione tra il generico valore del dataset e il corrispettivo valore di Labels, se fosse stato uno la moltiplicazione avrebbe restituito il valore altrimenti dava 0. Questo primo passaggio permetteva di ottenere in un registro solamente i valori del gruppo uno in modo tale da poterli sommare parzialmente.

Per il gruppo 0 si effettua un passaggio extra: Il registro che precedentemente conteneva la maschera veniva passato ad un'operazione di XOR con un registro di tutti valori pari ad uno; in questo modo si generava la maschera opposta (dove era presente uno si otteneva zero e viceversa) (l'operazione di XOR verrà fatta bit a bit tra i valori "0x00000000" e "0x00000001"). In questo modo si poteva ripetere il ragionamento applicato precedentemente con la moltiplicazione ed ottenere così le somme parziali dei valori del gruppo uno. Segue il codice.

```

ciclo_mediaGruppo:
cmp edi,ebx
jge residui_mediaGruppo
MOV esi, edi
IMUL esi, 16
MOVAPS XMM0, [ecx + esi] ;carico il quartetto associato alle Labels

MOVAPS XMM1, [edx + esi] ;carico il quartetto associato al Dataset | usato per gruppo 0
MOVAPS XMM2, XMM1 ;carico il quartetto associato al Dataset | usato per gruppo 1

MULPS XMM2, XMM0 ;in XMM2 ci sono i valori corrispondenti della maschera GRUPPO 1 * MASCHERA

XORPS XMM0, XMM5 ;Inversione della maschera

MULPS XMM1, XMM0 ;in XMM1 ci sono i valori corrispondenti alla maschera GRUPPO 0 * MASCHERA

ADDPS XMM3, XMM2 ;sommo i valori del gruppo 1 0|0|0|0 + num1|0|num2|0 + 0|num3|0|num4 =
ADDPS XMM4, XMM1 ;sommo i valori del gruppo 0
inc edi
jmp ciclo_mediaGruppo

residui_mediaGruppo:
HADDPS XMM3,XMM3
HADDPS XMM3,XMM3
HADDPS XMM4,XMM4
HADDPS XMM4,XMM4

```

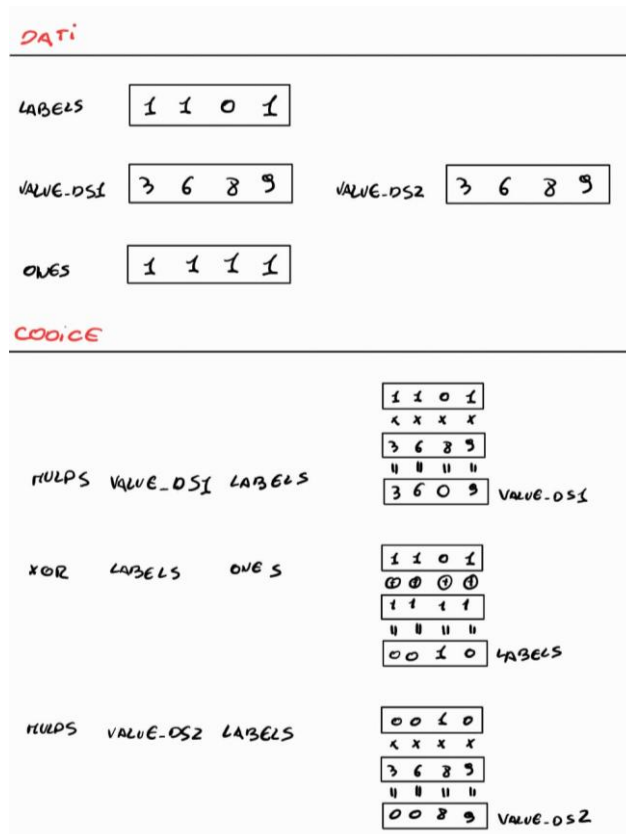


Figura 1 Illustrazione grafica del ragionamento implementato.

Anche per questa funzione si sono correttamente gestiti i residui, attraverso l'apposito ciclo dei residui. Il risultato anche qui viene passato per riferimento.

- A questo punto dello sviluppo progettuale, si è pensato di adoperare il **loop unrolling** nel codice C; più nello specifico, nella funzione **calcoloScore(...)**. Si è scelto un fattore di unroll pari a 4. Tale fattore andrebbe calibrato, e dunque scelto, in base alla componentistica hardware su cui si andrà ad eseguire il software; nonostante ciò, si è scelto di fissarlo pari a 4 in quanto si sono riscontrati tempi di elaborazione medi (calcolati sulle varie macchine dei componenti del gruppo) migliori per tale valore. Il fattore di unroll è stato definito mediante una define:

```
#define unrollFactor 4 //settiamo il fattore di unroll a 4
```

Il loop unrolling vero e proprio si può osservare all'interno della funzione **calcoloScore(...)**, come riportato nelle seguenti immagini:

```
type calcoloScore(int indice_feature_corrente, int indice_colonna, params* input)
{
    //aggiorniamo l'insieme di colonne S
    int k=indice_feature_corrente+1; //+1 perchè la cardinalità dell'insieme S è pari a k, ma dopo che aggiungiamo la
    colonna corrente per calcolare il nuovo score diventa k+1
    input->out[indice_feature_corrente]=indice_colonna; //S unione f_i (inseriamo la colonna corrente)

    //calcoliamo prima il numeratore
    //calcoliamo il point biserial correlation coefficient
    type rcf_assoluto=0.0;
    int i; //non lo inizializziamo nel for perchè l'indice "i" va condiviso con il for dei residui
    //visto che applichiamo il loop unrolling, decidiamo di far partire i dal fattore di unroll-1, così che gli indici
    dentro il for siano da i a i-unrollFactor. Questa scelta implica che nel ciclo dei residui dobbiamo fare i=
    (i-unrollFactor)+1. Se avessimo fatto partire il primo ciclo da i=unrollFactor, allora avremmo dovuto mettere i<=k,
    per prendere anche il caso di k%unrollFactor==0.
    for(i=unrollFactor-1;i<k;i+=unrollFactor) { //calcoliamo la media dei valori assoluti
        rcf_assoluto+=valoreAssoluto(pointBiserialCorrelationCoefficient(input, input->out[i-3]));
        rcf_assoluto+=valoreAssoluto(pointBiserialCorrelationCoefficient(input, input->out[i-2]));
        rcf_assoluto+=valoreAssoluto(pointBiserialCorrelationCoefficient(input, input->out[i-1]));
        rcf_assoluto+=valoreAssoluto(pointBiserialCorrelationCoefficient(input, input->out[i])); //ci fermiamo a 3
    }
    perchè l'unrollFactor è pari a 4
}
```

```

//ciclo residui
for(i=(i-unrollFactor)+1;i<k;i++) { //calcoliamo la media dei valori assoluti
    rcf_assoluto+=valoreAssoluto(pointBiserialCorrelationCoefficient(input, input->out[i]));
}
type numeratore=rcf_assoluto; //semplifichiamo il fatto che la media sia divisa per k, e si moltiplica per k

//calcoliamo ora il denominatore
//calcoliamo il coefficiente di correlazione di pearson tra la colonna corrente e le k-1 colonne già presenti
type rff_assoluto=0.0;
for(i=unrollFactor-1;i<k-1;i+=unrollFactor){//calcoliamo la media dei valori assoluti (a sto giro i si ferma prima
    perchè non vogliamo calcolare il coefficiente di pearson tra la colonna appena inserita e se stessa, ovviamente la
    colonna appena inserita è in coda)
    rff_assoluto+=valoreAssoluto(calcoloPearsonCoefficient(input, input->out[i-3], input->out
[indice_feature_corrente]));
    rff_assoluto+=valoreAssoluto(calcoloPearsonCoefficient(input, input->out[i-2], input->out
[indice_feature_corrente]));
    rff_assoluto+=valoreAssoluto(calcoloPearsonCoefficient(input, input->out[i-1], input->out
[indice_feature_corrente]));
    rff_assoluto+=valoreAssoluto(calcoloPearsonCoefficient(input, input->out[i], input->out
[indice_feature_corrente]));
}

//ciclo residui
for(i=(i-unrollFactor)+1;i<k-1;i++){//calcoliamo la media dei valori assoluti (a sto giro i si ferma prima perchè
non vogliamo calcolare il coefficiente di pearson tra la colonna appena inserita e se stessa, ovviamente la colonna
appena inserita è in coda)
    rff_assoluto+=valoreAssoluto(calcoloPearsonCoefficient(input, input->out[i], input->out
[indice_feature_corrente]));
}

//invece di calcolare l'rff tra tutte le coppie di colonne selezionate, lo calcoliamo tra le k-1 già presenti e la
k-esima appena inserita, e poi ne sommiamo l'rff passato che è una variabile globale che tiene traccia del calcolo tra
tutte le possibili coppie che ci manca
rff_assoluto+=rff_passato;

//aggiorniamo l'rff passato per la prossima volta che questa funzione verrà richiamata (in realtà poi deciderà la
funzione chiamante se renderlo permanente o meno in base al fatto se lo score calcolato adesso è il max tra tutti)
rff_passato_momentaneo=rff_assoluto;

//ora serve calcolare la media, dunque ci serve sapere il numero per cui dividere, tale numero lo calcoliamo
tramite la serie triangolare (esplicitata in funzione di k) (chiedere a Giovanni come funziona)
type den=((k-1)*k)/2;
type denominatore=(rff_assoluto/den)*(k)*(k-1); //stessa semplificazione di (k-1)/(k-1) di prima

denominatore+=k;

//eliminiamo la colonna dall'insieme

//eliminiamo la colonna dall'insieme
input->out[indice_feature_corrente]=-1;

//assembliamo
return numeratore/sqrt(denominatore);
}

```

Si noti come l'indicizzazione dei cicli è fortemente legata ad una scelta progettuale che comporta degli svantaggi e dei vantaggi accuratamente descritti nei commenti al codice. Ovviamente, sono stati necessari dei cicli per la gestione dei residui. Nonostante le previsioni iniziali, l'introduzione del loop unrolling non ha migliorato i tempi ma, anzi, si sono riscontrati dei piccoli ritardi. Ciò ci ha spinto a non

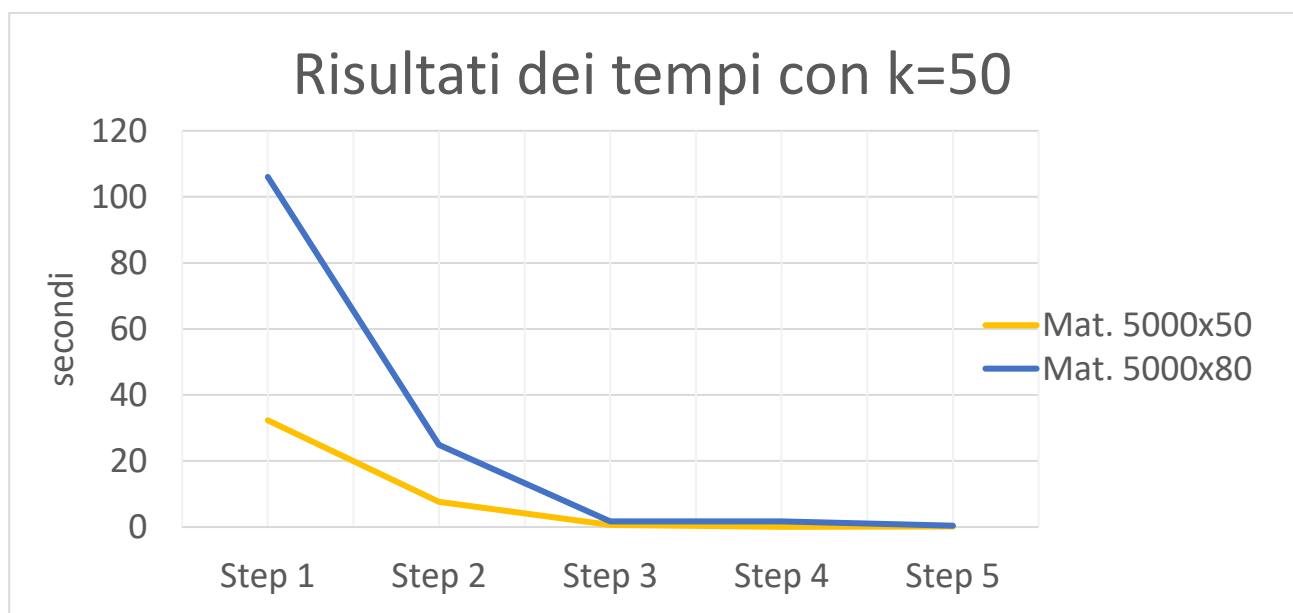
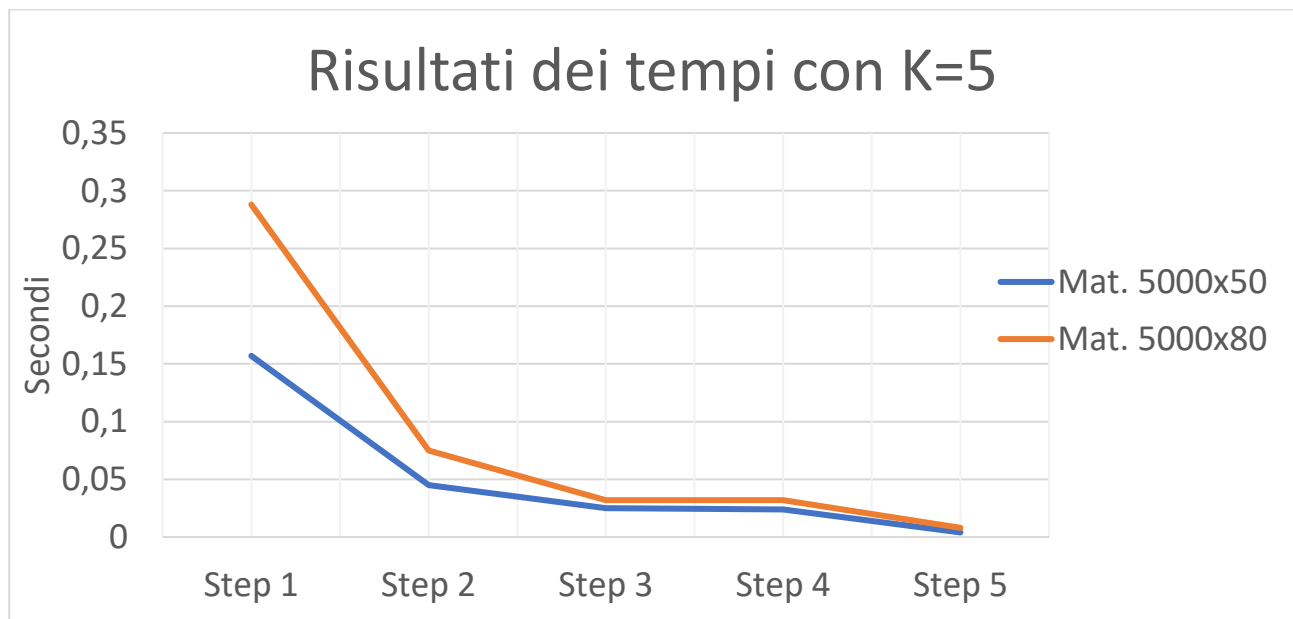
implementare tale modifica nel codice finale. Si approfitta di tale caso d'esempio per sottolineare, ancora una volta, l'utilità d'uso della piattaforma github; in particolare, grazie alla stessa siamo riusciti a ripristinare la versione subito precedente a tale modifica senza correre alcun rischio e, comunque, mantenendo "viva" la versione che includeva il loop unrolling. In conclusione, è doveroso esporre alcuni possibili motivi (che per vincoli di tempo e di strumenti non è stato possibile confermare o sfatare) del dovuto fallimento, in termini di prestazioni, del loop unrolling:

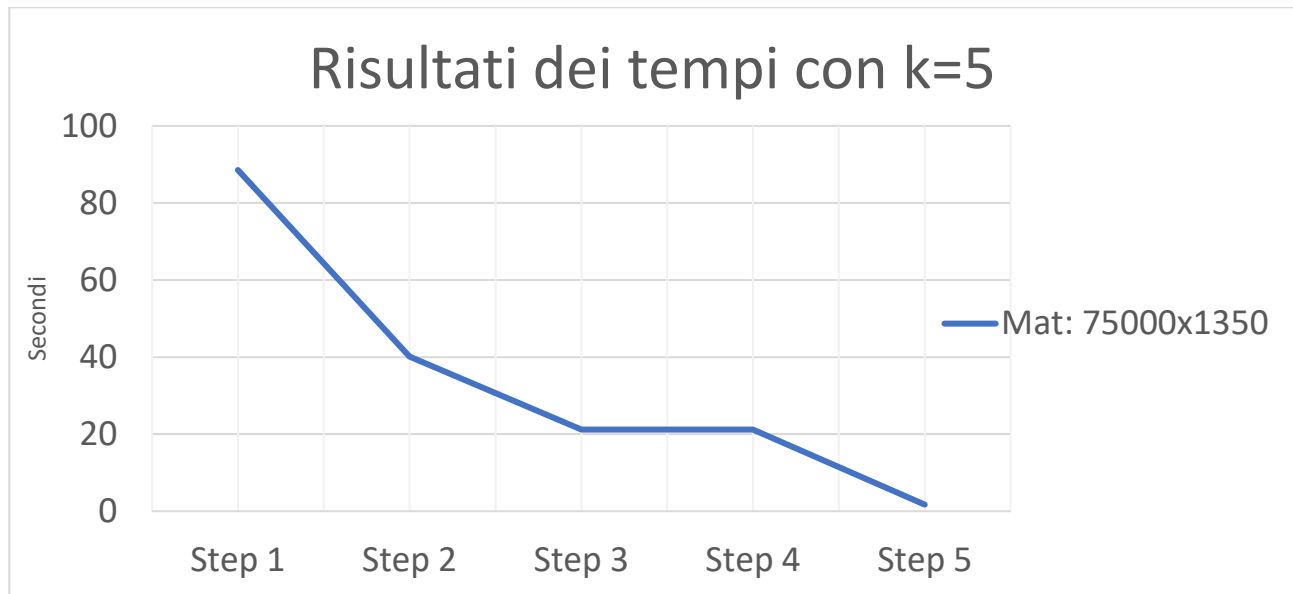
1. L'unrolling potrebbe aver portato a un aumento delle dimensioni del codice e, di conseguenza, a un aumento della probabilità di cache misses. Se il codice unrolled superasse la dimensione della cache, potrebbe risultare in un maggior numero di accessi alla memoria principale, influenzando negativamente le prestazioni.
2. L'espansione del loop potrebbe aver introdotto overhead aggiuntivo. Ad esempio, l'uso di più registri o un aumento della dimensione del codice possono portare a un incremento delle istruzioni di caricamento e archiviazione, aumentando potenzialmente il numero complessivo di istruzioni eseguite.
3. In alcuni casi, l'unrolling potrebbe introdurre inefficienze nel funzionamento della pipeline del processore. Ad esempio, una versione unrolled potrebbe portare a una maggiore probabilità di salti condizionati, interrompendo il flusso di istruzioni e causando un rallentamento della pipeline.
4. Alcuni compilatori possono avere difficoltà nell'analizzare e ottimizzare correttamente codice unrolled, specialmente se l'unrolling è fatto manualmente. Questo potrebbe portare a un peggioramento delle prestazioni invece di un miglioramento.

Da tali considerazioni si deduce che un'applicazione del loop unrolling è maggiormente consigliata nel codice Assembly del programma. Data la struttura dello stesso da noi seguita, è risultato troppo dispendioso ai fini del progetto stesso applicare una metodologa di loop unrolling direttamente al codice Assembly.

## Considerazioni

Dopo aver esaurito tutte le potenziali opportunità di miglioramento, ottimizzazione e traduzione di codice C in assembly, è stato elaborato un grafico per visualizzare l'andamento dei tempi dall'inizio dell'implementazione fino all'ultimo miglioramento.





In particolare:

- lo **Step 1** identifica i tempi riguardante l'esecuzione di solo codice C;
- lo **Step 2** identifica i tempi riguardante l'esecuzione del codice allo Step 1 + funzione calcoloSommatoriaQuadrato CalcoloMediaAssembly implementate in assembly;
- lo **Step 3** identifica i tempi riguardante l'esecuzione del codice ottimizzato allo Step 2 con modifica della funzione calcoloScore + funzione calcoloSommatoria in assembly;
- lo **Step 4** identifica i tempi riguardante l'esecuzione del codice ottimizzato allo Step 3 + la funzione calcolaValori\_nOn1 in assembly;
- lo **Step 5** identifica i tempi riguardante l'esecuzione del codice ottimizzato allo Step 4 + la funzione calcoloMediaGruppoAssembly in assembly.

Sono stati effettuati anche i test sulla piattaforma con risultati identici ma tempi più lenti, a seguire i valori ottenuti:

| TEST       | K | PIATTAFORMA | DISPOSITIVI PERSONALI |
|------------|---|-------------|-----------------------|
| 5000_80.ds | 5 | 0,21        | 0,008                 |

|               |    |       |        |
|---------------|----|-------|--------|
| 5000_80.ds    | 50 | 1,25  | 0,42   |
| 75000_1350.ds | 5  | 10,79 | 1,7320 |

A seguire i relativi screen della piattaforma:

```
Status: 0  
Time: 0.21  
  
sc: -1.000000  
  
0.016  
sc: 0.065544, out: [24,41,61,63,53,]
```

```
Status: 0  
Time: 10.79  
  
sc: -1.000000  
  
5.179  
sc: 0.051463, out: [1265,405,844,199,1337,]
```

```
Status: 0  
Time: 1.25  
  
sc: -1.000000  
  
1.063  
sc: 0.089538, out:
```



## Versione x86-64 AVX

### Codice in C

Per realizzare il progetto nella sua versione a 64 bit, si è deciso di iniziare con l'implementazione completa del codice in linguaggio C. In questa fase, il codice C a 32 bit nella sua versione altamente ottimizzata è stato adattato a 64 bit, apportando una semplice modifica al **#define type** passando da *float* a *double*.

Disponendo già del codice C nella sua forma altamente ottimizzata, nella seconda fase si è dovuto quindi solo procedere ad ottimizzare ulteriormente le diverse funzioni C mediante l'utilizzo di Assembly.

Le funzioni (le primitive matematiche) utilizzate sono identiche alla versione precedente, dunque sono presenti:

- Calcolo Sommatoria
- Calcolo Sommatoria Quadrato
- Calcolo Media
- Calcolo Media Gruppo
- Calcolo Valori n0 e n1
- Radice Quadrata in Assembly

L'implementazione di calcolo score non è stata modificata.

I tempi ottenuti tramite questo procedimento sono stati, come ci si aspettava, poco prestanti, a seguire i valori:

```
Dataset file name: 'test_5000_50_64.ds'  
Labels file name: 'test_5000_50_64.labels'  
Dataset row number: 5000  
Dataset column number: 50  
Number of features to extract: 5  
sc: -1.000000  
CFS time = 0.059 secs  
sc: 0.053390, out: [45,25,7,33,47,]
```

```
Dataset file name: 'test_75000_1350_64.ds'  
Labels file name: 'test_75000_1350_64.labels'  
Dataset row number: 75000  
Dataset column number: 1350  
Number of features to extract: 5  
sc: -1.000000  
CFS time = 32.657 secs  
sc: 0.051458, out: [1265,405,844,199,1337,]
```

```
Dataset file name: 'test_5000_80_64.ds'  
Labels file name: 'test_5000_80_64.labels'  
Dataset row number: 5000  
Dataset column number: 80  
Number of features to extract: 5  
sc: -1.000000  
CFS time = 0.097 secs  
sc: 0.065544, out: [24,41,61,63,53,]
```

```
Dataset file name: 'test_5000_50_64.ds'  
Labels file name: 'test_5000_50_64.labels'  
Dataset row number: 5000  
Dataset column number: 50  
Number of features to extract: 50  
sc: -1.000000  
CFS time = 2.282 secs  
sc: 0.063646, out: [45,25,7,33,47,22,5,31,26,18,6,11,9,27,44,38,32,8,42,14,10,49,29,43,30,48,41,19,0,16,24,21,37,1,13,35,17,15,3,40,39,23,28,46,2,20,12,36,34,4,]
```

```
Dataset file name: 'test_5000_80_64.ds'  
Labels file name: 'test_5000_80_64.labels'  
Dataset row number: 5000  
Dataset column number: 80  
Number of features to extract: 50  
sc: -1.000000  
CFS time = 6.256 secs  
sc: 0.089538, out: [24,41,61,63,53,76,78,26,18,57,62,11,30,34,70,25,16,29,8,37,13,22,49,10,45,9,46,54,27,55,14,19,21,74,42,39,77,23,71,50,68,67,65,40,66,31,32,20,79,12,]
```

## Implementazione funzione in Assembly: CalcoloMedia, CalcoloMediaGruppo, CalcoloSommatoria e CalcoloSommatoriaQuadrato

Le idee alla base delle implementazioni in Assembly sono pressoché invariate rispettivamente alla versione a 32bit.

Le principali differenze si riscontrano nei registri utilizzati in quanto si è passati dai registri XMM ai registri YMM in grado di contenere quattro valori, ciascuno di tipo Double. Un'altra differenza è il tipo di istruzioni utilizzate in quanto si è adottato il repertorio di istruzioni AVX.

Esaminiamo ora le implementazioni in Assembly, focalizzandoci su ciascuna per quanto riguarda l'implementazione del ciclo principale, il suo funzionamento e qualche particolare dovuto al repertorio AVX. Le fasi iniziale e finale, riguardanti il caricamento e il ritorno dei dati, sono dettate dalle convenzioni di chiamata (calling convention).

È importante notare che anche il calcolo degli elementi che dovrà essere gestito dal *ciclo dei residui* è uguale per tutte le funzioni (il calcolo è stato descritto nella versione a 32bit).

- ***extern void calcoloSommatoria(...)***, previo caricamento dati e inizializzazione dei vari registri, il codice è così implementato:

```
CSCiclo:
    cmp R15, R13 ; ripetiamo il ciclo numeroValori/4
    jge CSCicloFine
    mov R14, R15
    imul R14, 32
    vmovapd YMM2, [R9 + R14] ; prendo i successivi quattro valori della colonna1
    vmovapd YMM3, [R10 + R14] ; prendo i successivi quattro valori della colonna2
    vsubpd YMM2, YMM4 ; input->ds[indiceColonna1 + (i * input->d)] - mediaColonna1
    vsubpd YMM3, YMM5 ; input->ds[indiceColonna2 + (i * input->d)] - mediaColonna2
    vmulpd YMM2, YMM3 ; sto facendo il prodotto dei due fattori. Il risultato viene salvato in XMM0
    vaddpd YMM6, YMM2 ; sto facendo la somma per adesso di 4 a 4 valori, solo alla fine verranno sommati tutti i 4
    inc R15
    jmp CSCiclo
```

Si utilizzano i registri YMM2 e YMM3 per caricare rispettivamente i dati dalla colonna1 e dalla colonna2 interessate. A seguire si effettua una sottrazione con i valori medi, rispettivi passati come parametri e caricati nei registri YMM4 e YMM5 precedentemente. Infine, si fa la moltiplicazione per calcolare il quadrato e la somma per implementare la sommatoria.

```

CSCicloFine:
    imul R15, 4

    ; effettuo le due somme parziali per sommare i 4 valori del registro XMM3
    vhaddpd YMM6, YMM6, YMM6
    vextractf128 XMM7, YMM6, 0
    vextractf128 XMM8, YMM6, 1
    vaddpd XMM0, XMM7, XMM8
    vxorps XMM6, XMM6
    mov R13D, [RDI + 36]

CSCiclo2:
    cmp R15, R13
    jge CSCicloFine2
    mov R14, R15
    add R14, 8

    vmovsd XMM2, [R9 + R14] ; prendo il successivo singolo valore della colonna1
    vmovsd XMM3, [R10 + R14] ; prendo il successivo singolo valore della colonna2
    vsubsd XMM2, XMM4 ; input->ds[indiceColonna1 + (i * input->d)] - mediaColonna1
    vsubsd XMM3, XMM5 ; input->ds[indiceColonna2 + (i * input->d)] - mediaColonna2
    vmulsd XMM2, XMM3 ; sto facendo il prodotto dei due fattori. Il risultato viene salvato in YMM2
    vaddsd XMM6, XMM2 ; sto facendo la somma per adesso di 4 a 4 valori, solo alla fine verranno sommati tutti i 4

    inc R15
    jmp CSCiclo2

```

Notiamo immediatamente un particolare utilizzo dell'istruzione *vhaddpd*. Ciò è dovuto (dopo una attenta analisi) al fatto che la somma orizzontale del registro YMM non produceva un output simile a quello dell'istruzione *haddps* di SSE (versione a 32bit). In altre parole, le somme parziali orizzontali non vengono memorizzate nelle posizioni prima/terza e seconda/quarta (come avviene con l'output di *haddps*), ma piuttosto nelle posizioni prima/seconda e terza/quarta. Di conseguenza, per ottenere i valori corretti, è stato necessario estrarli singolarmente tramite la funzione *vextractf128*. Questa implementazione è adottata in tutte le funzioni sviluppate in Assembly.

- **extern void calcoloSommatoriaQuadrato(...)**, lo sviluppo della funzione è molto simile alla precedente, la differenza più importante viste le sue specifiche la si evince all'interno del ciclo:

```
CSQCiclo:
    cmp R15, R13 ; ripetiamo il ciclo numeroValori/4
    jge CSQCicloFine
    mov R14, R15
    imul R14, 32

    vmovapd YMM1, [R9 + R14] ; prendo i successivi quattro valori
    vmovapd YMM2, YMM1 ; copio i quattro valori
    vsubpd YMM1, YMM3 ; input->ds[indiceColonna + (i * input->d)] - mediaValoriColonna
    vsubpd YMM2, YMM3 ; input->ds[indiceColonna + (i * input->d)] - mediaValoriColonna
    vmulpd YMM1, YMM2 ; sto facendo il quadrato. Il risultato viene salvato in YMM1
    vaddpd YMM4, YMM1 ; sto facendo la somma per adesso di 4 a 4 valori, solo alla fine verranno sommati tutti i 4

    inc R15

    jmp CSQCiclo
```

dovendo fare i quadrato dei valori della i-esima colonna, allora i quattro valori contigui della i-esima colonna vengono caricati in copia su due registri diversi quali YMM1 e YMM2. Successivamente, si effettua la somma con il valore mediaValoriColonna. Di seguito il quadrato, i quadrati ottenuti vengono sommati in YMM4 come somme parziali.

```
CSQCicloFine:
    imul R15, 4

    ; effettuo le due somme parziali per sommare i 4 valodi del registro XMM3
    vhaddpd YMM4, YMM4, YMM4
    vextractf128 XMM5, YMM4, 0
    vextractf128 XMM6, YMM4, 1
    vaddpd XMM0, XMM5, XMM6

    mov R13D, [RDI + 36]

    vxorps XMM4, XMM4

CSQCiclo2:
    cmp R15, R13
    jge CSQCicloFine2
    mov R14, R15
    add R14, 8

    vmovsd XMM1, [R9 + R14] ; prendo il successivo valore
    vmovsd XMM2, XMM1 ; copio il singolo valore
    vsubsd XMM1, XMM3 ; input->ds[indiceColonna + (i * input->d)] - mediaValoriColonna
    vsubsd XMM2, XMM3 ; input->ds[indiceColonna + (i * input->d)] - mediaValoriColonna
    vmulsd XMM1, XMM2 ; sto facendo il quadrato. Il risultato viene salvato in XMM0
    vaddsd XMM4, XMM1 ; sto facendo la somma per adesso di 4 a 4 valori, solo alla fine verranno sommati tutti i 4

    inc R15

    jmp CSQCiclo2
```

Notare l'adozione di *vhaddpd* e *vextractf128* come prima discusso.

- ***extern void calcoloMedia(...)***, previo caricamento dei dati e inizializzazione dei registri impiegati il calcolo della media è così sviluppato. Non ci sono dettagli da attenzionare visto la medesima implementazione come per 32bit.

```
ciclo_calcoloMedia:
    cmp r11,r15
    jge calcoloMediaResidui
    MOV r12,r11
    IMUL r12,32
    vmovapd ymm0,[rbx+r12]

    inc r11
    cmp r11,r15
    jge calcoloMediaResidui

    mov r12,r11
    IMUL r12,32
    vmovapd ymm1,[rbx+r12]
    vaddpd ymm2,ymm0
    vaddpd ymm2,ymm1

    inc r11
    jmp ciclo_calcoloMedia
```

```
calcoloMediaResidui:
    vhaddpd ymm2,ymm2
    vextractf128 xmm8,ymm2,0
    vextractf128 xmm9,ymm2,1
    vaddpd xmm8,xmm9 ;c'è la somma dei valori

    imul r11,4
    mov r15d,[rdi+36]

    vxorps ymm2, ymm2

ciclo_calcoloMediaResidui:
    cmp r11, r15
    jge fine_Media
    mov r14, r11
    imul r14, 8

    vmovsd xmm1, [rbx + r14] ; prendo il successivo valore
    vaddsd xmm2, xmm1
    inc r11
    mov r14,r11
    shr r14,2

    jmp ciclo_calcoloMediaResidui
```

Il calcolo dei residui avviene in maniera medesima ai casi analizzati precedentemente con sempre lo stesso problema legato alla somma parallela tramite istruzione *vhaddpd*.

- ***extern void calcoloMediaGruppo(...)***, si è optato di effettuare direttamente la differenza tra le due medie in Assembly per avere così in C direttamente il valore desiderato e non dover apportare operazioni extra. L'idea alla base è identica alla

versione a 32bit, dunque si hanno diversi registri di supporto ed in particolare un registro riempito da valore 1 in modo tale che si possa effettuare lo XOR sulla maschera dei Labels. I valori vengono caricati quattro alla volta su due registri separati, uno dedicato al calcolo del gruppo 1, mentre l'altro è dedicato al calcolo del gruppo 0. Infine, le somme parziali vengono inserite negli appositi registri. Segue il codice:

```
ciclo_mediaGruppo:
    cmp r11,r15
    jge residui_mediaGruppo
    MOV r12,r11
    IMUL r12,32
    vmovapd YMM0,[r10+r12] ;caricamento valori label
    vmovapd YMM1,[rbx+r12] ;caricamento dei dati del dataset
    vmovapd YMM2,YMM1

    vmulpd YMM1,YMM0
    vxorpd YMM0,YMM5
    vmulpd YMM2,YMM0

    vaddpd YMM3,YMM1 ;somma gruppo 1
    vaddpd YMM4,YMM2 ;somma gruppo 0

    inc r11
    jmp ciclo_mediaGruppo
```

A seguire si ha il calcolo di eventuali residui.

```
ciclo_calcoloMediaGruppoResidui:
    cmp r11, r15
    jge fine_MediaGruppo
    mov r14, r11
    imul r14, 8

    vmovsd xmm1, [rbx + r14] ; prendo il successivo valore
    vaddsd xmm2, xmm1

    vmovsd xmm0, [r10+r14] ; caricamento valori label
    vmovsd xmm1, [rbx+r14] ; caricamento dei dati del dataset
    vmovsd xmm2, xmm1

    vmulsd xmm1, xmm0
    pxor xmm0, xmm5
    vmulsd xmm2, xmm0

    vaddsd xmm8, xmm1 ; somma gruppo 1
    vaddpd xmm10, xmm2 ; somma gruppo 0

    inc r11
    jmp ciclo_calcoloMediaGruppoResidui
```

I test ottenuti con queste modifiche portano ad evidenti ottimizzazioni e un decremento notevole dei tempi:



|                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>Dataset file name: 'test_5000_50_64.ds' Labels file name: 'test_5000_50_64.labels' Dataset row number: 5000 Dataset column number: 50 Number of features to extract: 5 sc: -1.000000  CFS time = 0.005 secs sc: 0.053390, out: [45,25,7,33,47,]</pre> | <pre>Dataset file name: 'test_75000_1350_64.ds' Labels file name: 'test_75000_1350_64.labels' Dataset row number: 75000 Dataset column number: 1350 Number of features to extract: 5 sc: -1.000000  CFS time = 1.936 secs sc: 0.051458, out: [1265,405,844,199,1337,]</pre> | <pre>Dataset file name: 'test_5000_80_64.ds' Labels file name: 'test_5000_80_64.labels' Dataset row number: 5000 Dataset column number: 80 Number of features to extract: 5 sc: -1.000000  CFS time = 0.008 secs sc: 0.065544, out: [24,41,61,63,53,]</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>Dataset file name: 'test_5000_80_64.ds' Labels file name: 'test_5000_80_64.labels' Dataset row number: 5000 Dataset column number: 80 Number of features to extract: 50 sc: -1.000000  CFS time = 0.449 secs sc: 0.089538, out: [24,41,61,63,53,76,78,26,18,57,62,11,30,34,70,25,16,29,8,37,13,22,49,10,45,9,46,54,27,55,14,19,21,74,42,39,77,23,71,50,68,67,65,40,66,31,32,20,79,12,]</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>Dataset file name: 'test_5000_50_64.ds' Labels file name: 'test_5000_50_64.labels' Dataset row number: 5000 Dataset column number: 50 Number of features to extract: 50 sc: -1.000000  CFS time = 0.168 secs sc: 0.063646, out: [45,25,7,33,47,22,5,31,26,18,6,11,9,27,44,38,32,8,42,14,10,49,29,43,30,48,41,19,0,16,24,21,37,1,13,35,17,15,3,40,39,23,28,46,2,20,12,36,34,4,]</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Implementazione funzione in Assembly: sqrtAssembly e calcoloValori\_n0n1

Le ultime due funzioni inserite per l'ottimizzazione sono state il calcolo della radice quadrata in assembly e l'inserimento del calcolo dei valori di n0 e n1 utili nei successivi calcoli.

- **extern void sqrtAssembly (...),**

```
sqrtAssembly:
    ; sequenza di ingresso nella funzione
    push rbp ; salva il Base Pointer
    mov rbp, rsp ; il Base Pointer punta al Record di Attivazione corrente

    ; lettura dei parametri dal Record di Attivazione
    ; in xmm0 c'è il parametro di tipo double

    ; corpo della funzione
    sqrtsd xmm0, xmm0 ; calcoliamo la radice quadrata
    ; il valore viene restituito nei registri xmm0 e xmm1

    ; sequenza di uscita dalla funzione

    mov rsp, rbp ; ripristina lo Stack Pointer
    pop rbp ; ripristina il Base Pointer
    ret ; ritorna alla funzione chiamant
```

- **extern void calcoloValori\_n0n1(...),** per effettuare la somma si è optato di effettuare prima il calcolo di tutti i valori di 1 andando a calcolare i valori prendendone quattro alla volta, effettuando la somma al termine del ciclo avremo nel registro YMM0 tutte le somme parziali del valore uno in quanto lo zero non apporta modifiche alla somma.

```
ciclo_calcoloValore0n1:
    cmp edx, eax
    JGE fine_ciclo_calcoloValore0n1
    vmovapd ymm1, [rbx+r10*8] ; prendiamo 4 labels da 64 bit, quindi il prossimo indirizzo utile sta a 8 byte di distanza
    vaddpd ymm0, ymm1 ; le sommiamo alle vecchie
    add edx, 4 ; abbiamo appena preso 4 valori da 64 bit, quindi incrementiamo
    ; il contatore di 4.
    add r10, 4
    JMP ciclo_calcoloValore0n1 ; chiudiamo il for
```

```

fine_ciclo_calcoloValoren0n1:

;usiamo la riduzione prima di calcolare i residui
vhaddpd ymm0, ymm0
;ora dovremmo sommare i 64 bit più significativi con i 64 meno significativi,
;per farlo procediamo come dis eguito

; Estrai i 128 bit più significativi da ymm0
vextractf128 xmm1, ymm0, 1

;sommiamo ciò detto prima
addpd xmm0, xmm1

;potrebbero esserci dei residui da calcolare
cmp edx, eax
JE fine_residui_calcoloValoren0n1 ;se r8==rax allora non possiamo avere residui
sub edx, 3 ;ripristiniamo il contatore
sub r10, 3

residui_calcoloValoren0n1: ;esattamente come il for di sopra ma prende elementi da 64 bit singoli
    CMP edx, eax
    JGE fine_residui_calcoloValoren0n1
    movsd xmm1, [rbx+r10*8]
    addsd xmm0, xmm1
    add edx, 1
    add r10, 1

    JMP residui_calcoloValoren0n1

```

Si effettua il calcolo sugli eventuali residui presenti sempre applicando lo stesso ragionamento visto prima e andando a sommare i valori pari ad 1.

Con questo procedimento al termine di entrambi i cicli avremo nel registro xmm0 l'intera somma dei valori pari ad 1.

```

cvtsd2si ecx, xmm0 ;converto il valore di n1 in intero
sub eax, ecx ;calcoliamo il valore di n0 (rax>=rcx sempre)

```

Per ottenere la quantità di valori pari a 0 è sufficiente eseguire la seguente differenza:

$$Totale - Quantità\ valori\ 1$$

I tempi ottenuti inserendo queste ultime due modifiche non apportano una notevole diminuzione dei tempi.

|                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dataset file name: 'test_5000_50_64.ds'<br>Labels file name: 'test_5000_50_64.labels'<br>Dataset row number: 5000<br>Dataset column number: 50<br>Number of features to extract: 5<br>sc: -1.000000<br><br>CFS time = 0.005 secs<br>sc: 0.053390, out: [45,25,7,33,47,]                                                                                                                                        | Dataset file name: 'test_5000_80_64.ds'<br>Labels file name: 'test_5000_80_64.labels'<br>Dataset row number: 5000<br>Dataset column number: 80<br>Number of features to extract: 5<br>sc: -1.000000<br><br>CFS time = 0.009 secs<br>sc: 0.065544, out: [24,41,61,63,53,] | Dataset file name: 'test_75000_1350_64.ds'<br>Labels file name: 'test_75000_1350_64.labels'<br>Dataset row number: 75000<br>Dataset column number: 1350<br>Number of features to extract: 5<br>sc: -1.000000<br><br>CFS time = 1.845 secs<br>sc: 0.051458, out: [1265,405,844,199,1337,] |
| Dataset file name: 'test_5000_80_64.ds'<br>Labels file name: 'test_5000_80_64.labels'<br>Dataset row number: 5000<br>Dataset column number: 80<br>Number of features to extract: 50<br>sc: -1.000000<br><br>CFS time = 0.435 secs<br>sc: 0.089538, out: [24,41,61,63,53,76,78,26,18,57,62,11,30,34,70,25,16,29,8,37,13,22,49,10,45,9,46,54,27,55,14,19,21,74,42,39,77,23,71,50,68,67,65,40,66,31,32,20,79,12,] |                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                          |
| Dataset file name: 'test_5000_50_64.ds'<br>Labels file name: 'test_5000_50_64.labels'<br>Dataset row number: 5000<br>Dataset column number: 50<br>Number of features to extract: 50<br>sc: -1.000000<br><br>CFS time = 0.173 secs<br>sc: 0.063646, out: [45,25,7,33,47,22,5,31,26,18,6,11,9,27,44,38,32,8,42,14,10,49,29,43,30,48,41,19,0,16,24,21,37,1,13,35,17,15,3,40,39,23,28,46,2,20,12,36,34,4,]         |                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                          |

Tuttavia, è da notare che il test effettuato sul dataset di dimensioni 75000\*1350 (test 3 della piattaforma) ha registrato una riduzione del tempo di circa 0.200 secondi rispetto alla versione senza l'implementazione delle funzioni *calcoloValori\_nOn1* e *sqrtAssembly* in Assembly. Ciò suggerisce che queste funzioni potrebbero migliorare le prestazioni, soprattutto quando il dataset utilizzato è di dimensioni considerevoli visto che negli altri dataset di dimensioni ridotti tale guadagno non si è apprezzato.

Per confermare questa ipotesi, è stato eseguito nuovamente il test sullo stesso dataset, ma con un valore di k più grande pari a quindici. Di seguito sono riportati i risultati, prima senza l'implementazione in Assembly delle funzioni *calcoloValori\_nOn1* e *sqrtAssembly* e poi con:

```
Dataset file name: 'test_75000_1350_64.ds'
Labels file name: 'test_75000_1350_64.labels'
Dataset row number: 75000
Dataset column number: 1350
Number of features to extract: 15
sc: -1.000000

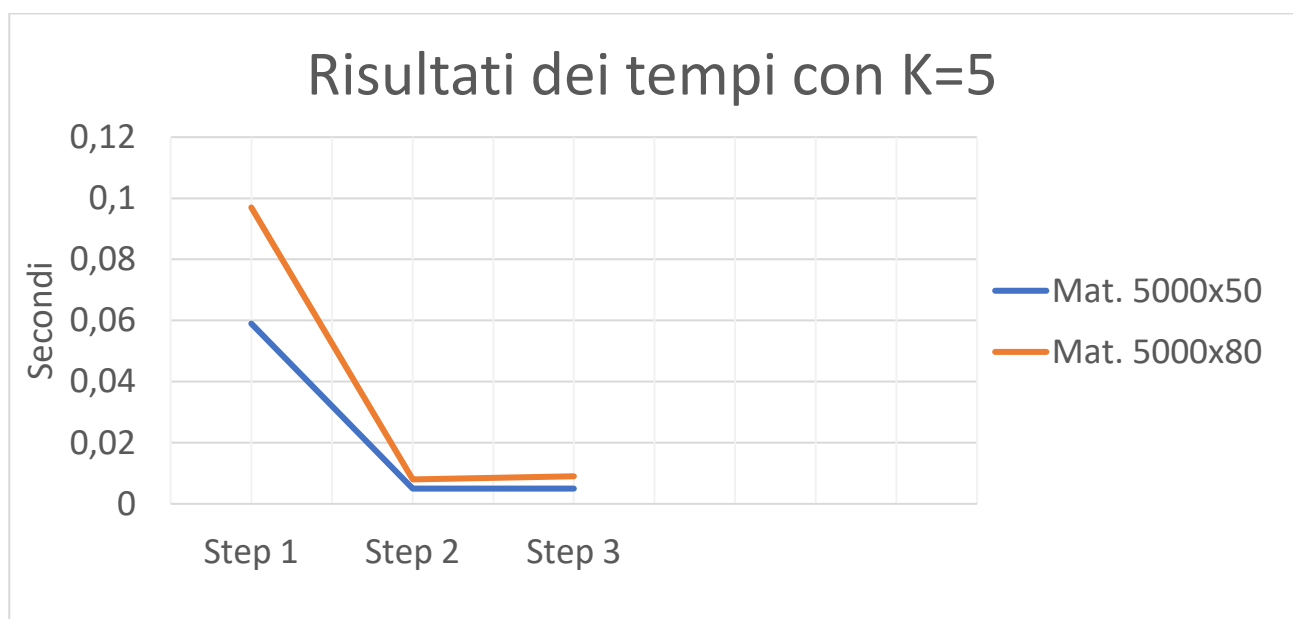
CFS time = 32.585 secs
sc: 0.079748, out: [1265,405,844,199,1337,45,13,328,629,1320,918,855,358,1345,265,]
```

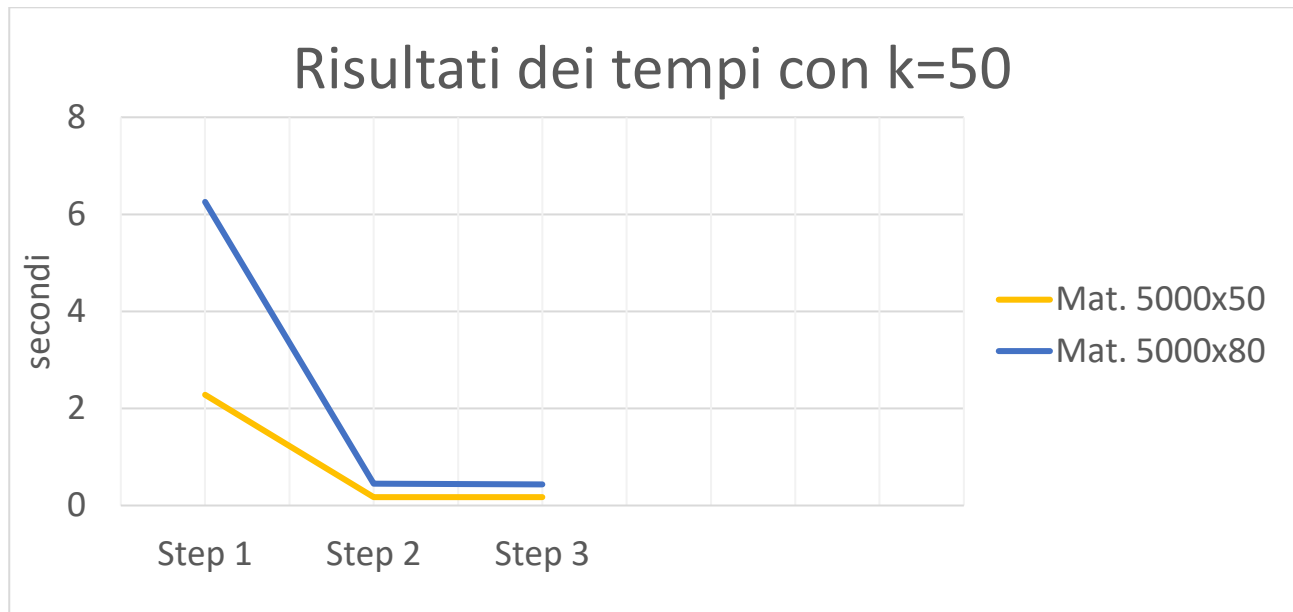
```
Dataset file name: 'test_75000_1350_64.ds'  
Labels file name: 'test_75000_1350_64.labels'  
Dataset row number: 75000  
Dataset column number: 1350  
Number of features to extract: 15  
sc: -1.000000  
  
CFS time = 31.708 secs  
sc: 0.079748, out: [1265,405,844,199,1337,45,13,328,629,1320,918,855,358,1345,265,]
```

Come evidenziato, si è ottenuto un risparmio di tempo di circa un secondo. Tuttavia, non si esclude che la differenza di tempo osservata sia influenzata dai processi in corso sulla macchina.

### Considerazioni

Di seguito vengono mostrati tre grafici che illustrano la differenza di prestazione in termini di tempo ottenute al seguito della implementazione del codice a x64.





In particolare:

- lo **Step 1** identifica i tempi riguardante l'esecuzione di solo codice C con le varie ottimizzazioni ottenute durante la fase di sviluppo della software a 32bit;
- lo **Step 2** identifica i tempi riguardante l'esecuzione del codice allo Step 1 + tutte le funzioni implementate in assembly;
- lo **Step 3** identifica i tempi riguardante l'esecuzione del codice ottimizzato allo Step 2 + l'implementazione in assembly della funzione `sqrAssembly()` e `calcoloValori_non1()`;

Sono stati effettuati anche i test sulla piattaforma con risultati identici ma tempi più lenti, a seguire i valori ottenuti:

| TEST          | K  | PIATTAFORMA | DISPOSITIVI PERSONALI |
|---------------|----|-------------|-----------------------|
| 5000_80.ds    | 5  | 0,23        | 0,009                 |
| 5000_80.ds    | 50 | 1,489       | 0,435                 |
| 75000_1350.ds | 5  | 7,526       | 1,845                 |

A seguire i relativi screen della piattaforma:

```
Status: 0
Time: 0.23

sc: -1.000000

0.023
sc: 0.065544, out: [24,41,61,63,53,]
```

```
Status: 0
Time: 1.69

sc: -1.000000

1.489
sc: 0.089538, out: [24,41,61,63,53,76,78,26,18,57,62,11,30,34,70,25,16,29,8,37,13,22,49,10,45,9,46,54,27,55,14,19,21,74,42,39,77,23,71,50,68,67,65,40,66,31,32,20,79,12,]
```

```
Status: 0
Time: 15.73

sc: -1.000000

7.526
sc: 0.051458, out: [1265,405,844,199,1337,]
```

## OpenMp

### Versione a 32 bit e 64 bit

Nel corso di questo progetto, abbiamo integrato la libreria OpenMP (nel codice che fa solo uso delle funzioni scritte in C, escludendo le funzioni scritte in Assembly) per sfruttare il

Danilo Fortugno Mat. xxxxxx      Giovanni Pascuzzi Mat. xxxxxx      Marco Ziccardi Mat. xxxxxx

potenziale del parallelismo in alcune sezioni critiche del codice. L'utilizzo di OpenMP è stato mirato a migliorare le prestazioni computazionali, consentendo l'esecuzione simultanea di determinate operazioni su più thread. Nello specifico, abbiamo identificato sezioni di codice in cui il parallelismo potrebbe portare a benefici tangibili, come cicli for lunghi o operazioni indipendenti su grandi set di dati. L'integrazione di OpenMP è stata implementata attraverso l'inserimento di direttive pragma OpenMP, che consentono al compilatore di generare codice parallelo. Abbiamo adottato strategie di gestione delle variabili condivise, utilizzando clausole come "reduction" per garantire la coerenza e l'integrità dei dati condivisi tra i thread. L'obiettivo è stato ottimizzare l'utilizzo delle risorse hardware disponibili, accelerando l'esecuzione delle porzioni di codice critico. L'analisi delle prestazioni è stata condotta per valutare l'impatto della parallelizzazione sulla velocità di esecuzione del programma e per assicurare che i benefici ottenuti fossero proporzionati all'overhead introdotto dalla gestione di thread paralleli. L'integrazione di OpenMP è stata una componente significativa della nostra strategia di ottimizzazione, contribuendo a massimizzare l'efficienza computazionale e a garantire una migliore scalabilità del nostro software su architetture multicore. Di seguito sono riportate le modifiche apportate alle funzioni ***calcoloScore(...)***, ***calcoloMediaGruppo(...)***, ***calcoloSommatoria(...)***, ***calcoloSommatoriaQuadrato(...)*** e ***calcoloMedia(...)***:

```
type calcoloScore(int indice_feature_corrente, int indice_colonna, params* input)
{
    //aggiorniamo l'insieme di colonne S
    int k=indice_feature_corrente+1; //+1 perch  la cardinalit  dell'insieme S   pari a k, ma dopo che aggiungiamo la colonna corre
    input->out[indice_feature_corrente]=indice_colonna; //S unione f_i (inseriamo la colonna corrente)

    //calcoliamo prima il numeratore
    //calcoliamo il point biserial correlation coefficient
    type rcf_assoluto=0.0;
    #pragma omp parallel for reduction(+:rcf_assoluto)
    for(int i=0;i<k;i++) { //calcoliamo la media dei valori assoluti
        rcf_assoluto+=valoreAssoluto(pointBiserialCorrelationCoefficient(input, input->out[i]));
    }
    type numeratore=rcf_assoluto; //semplifichiamo il fatto che la media sia divisa per k, e si moltiplica per k

    //calcoliamo ora il denominatore
    //calcoliamo il coefficiente di correlazione di pearson tra la colonna corrente e le k-1 colonne gi  presenti
    type rff_assoluto=0.0;
    #pragma omp parallel for reduction(+:rff_assoluto)
    for(int i=0;i<k-1;i++) { //calcoliamo la media dei valori assoluti (a sto giro i si ferma prima perch  non vogliamo calcolare il
        rff_assoluto+=valoreAssoluto(calcoloPearsonCoefficient(input, input->out[i], input->out[indice_feature_corrente]));
    }
}
```



```

type calcoloMediaGruppo(params *input, int j) {
    // Inizializzazione dei Dati
    type somma0 = 0;
    type somma1 = 0;

    #pragma omp parallel for reduction(+:somma0, somma1)
    for (int i = 0; i < input->N; i++) {
        if (0 == input->labels[i]) {
            somma0 += input->ds[(j * input->N) + i]; // perch  si ha la matriche come vettore riga
        } else {
            somma1 += input->ds[(j * input->N) + i]; // perch  si ha la matriche come vettore riga
        }
    }

    // calcolo Media
    return (somma0 / n0) - (somma1 / n1);
}

```

```

type calcoloSommatoria(params *input, type mediaColonna1, type mediaColonna2, int indiceColonna1, int indiceColonna2) {
    // inizializziamo la somma a 0 perch  l'identit  per l'addizione
    type somma = 0;

    #pragma omp parallel for reduction(+:somma)
    for (int i = 0; i < input->N; i++) {
        somma += (input->ds[(indiceColonna1 * input->N) + i] - mediaColonna1) * (input->ds[(indiceColonna2 * input->N) + i] - mediaCo
    }

    // restituzione risultati
    return somma;
}

```

```

type calcoloSommatoriaQuadrato(params *input, type mediaValoriColonna, int indiceColonna) {
    // inizializziamo la somma a 0 perch  l'identit  per l'addizione
    type somma = 0;

    #pragma omp parallel for reduction(+:somma)
    for (int i = 0; i < input->N; i++) {
        somma += calcoloQuadrato(input->ds[(indiceColonna * input->N) + i] - mediaValoriColonna);
    }

    return somma;
}

```

```

type calcoloMedia(params *input, int j) {
    // Inizializzazione dei Dati
    float somma = 0;

    #pragma omp parallel for reduction(+:somma)
    for (int i = 0; i < input->N; i++) {
        somma += input->ds[(j * input->N) + i];
    }

    // calcolo Media
    return somma / input->N;
}

```

## Considerazioni

In seguito all'implementazione di OpenMP si sono osservati tempi di esecuzione pressoch  invariati, se non peggiori, sia su dataset "piccoli" e sia su dataset "grandi" (75000x1350) con  $k=5$ ; in particolare, senza l'utilizzo di OpenMP si ottiene:

```
➤ $./runcfs32 -ds test_75000_1350_32.ds -labels test_75000_1350_32.labels -k 5 -d  
Dataset file name: 'test_75000_1350_32.ds'  
Labels file name: 'test_75000_1350_32.labels'  
Dataset row number: 75000  
Dataset column number: 1350  
Number of features to extract: 5  
CFS time = 35.813 secs  
sc: 0.051457, out: [1265,405,844,199,1337,]  
Done.
```

Invece

utilizzando

OpenMP:

```
➤ $./runcfs32 -ds test_75000_1350_32.ds -labels test_75000_1350_32.labels -k 5 -d  
Dataset file name: 'test_75000_1350_32.ds'  
Labels file name: 'test_75000_1350_32.labels'  
Dataset row number: 75000  
Dataset column number: 1350  
Number of features to extract: 5  
CFS time = 36.757 secs  
sc: 0.051457, out: [1265,405,844,199,1337,]  
Done.
```